CS 584 Natural Language Processing

Yue Ning
Department of Computer Science
Stevens Institute of Technology

Lecture 4: Recurrent Neural Networks and Beyond

Learning Objectives

- Understand RNN Language Models
- Understand CNN Applications

Today’s lecture

- From N-gram to RNN
- Vanishing gradient -> LSTM and others
- CNN for text
- Tokenization

Part 1

From N-gram to RNN

N-gram language models in practice

- You can build a simple tri-gram language model over a 1.7 million word corpus (Reuters) in a few seconds on your laptop*
- today the ___

company	0.153
bank	0.153
price	0.077
italian	0.039
emirate	0.039

Sparsity problem: not much granularity in the probability distribution

Otherwise, seems reasonable!

* https://nlpforhackers.io/language-models/

Generating text with a n-gram language model

- You can also use a language model to generate text.

today the ___

Condition on this

company	0.153
bank	0.153
price	0.077
italian	0.039
emirate	0.039

sample

* https://nlpforhackers.io/language-models/

Generating text with a n-gram language model

- You can also use a language model to generate text.

today the price ___

Condition on this

of	0.308
for	0.050
it	0.146
to	0.046
is	0.031
...	

sample

* https://nlpforhackers.io/language-models/

Generating text with a n-gram language model

- You can also use a language model to generate text.

today the price of ___

Condition on this

the	0.072
18	0.043
oil	0.043
its	0.036
gold	0.018
...	

sample

Sparsity issue:

P(gold | price of) = 0.018
P(oil | price of) = 0.043
P(happiness | price of) = 0

* https://nlpforhackers.io/language-models/

Generating text with a n-gram language model

- You can use a language model to generate text: today the price of gold___
  - Result: *today the price of gold per ton, while production of shoe lasts and shoe industry, the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks, sept 30 end primary 76 cts a share*
- Surprisingly grammatical!

- Incoherent!: we need to consider more than 3 words at a time if we want to generate good text.
- But increasing n worsens sparsity problem, and increase model size…

How to build a neural language model?

- Recall the language model task:
  - input: sequence of words $x^{(1)},x^{(2)},\ldots,x^{(t)}$
  - output: prob dist of the next word
    - $P(x^{(t+1)}|x^{(t)},\ldots,x^{(1)})$
- How about a window-based neural model?

~~as the proctor started the clock~~ the students opened their _____

discard

fixed window

A fixed-window neural language model

- output distribution

$$\hat{y} = \text{softmax}(Uh + b_2) \in R^{|V|}$$

- hidden layer

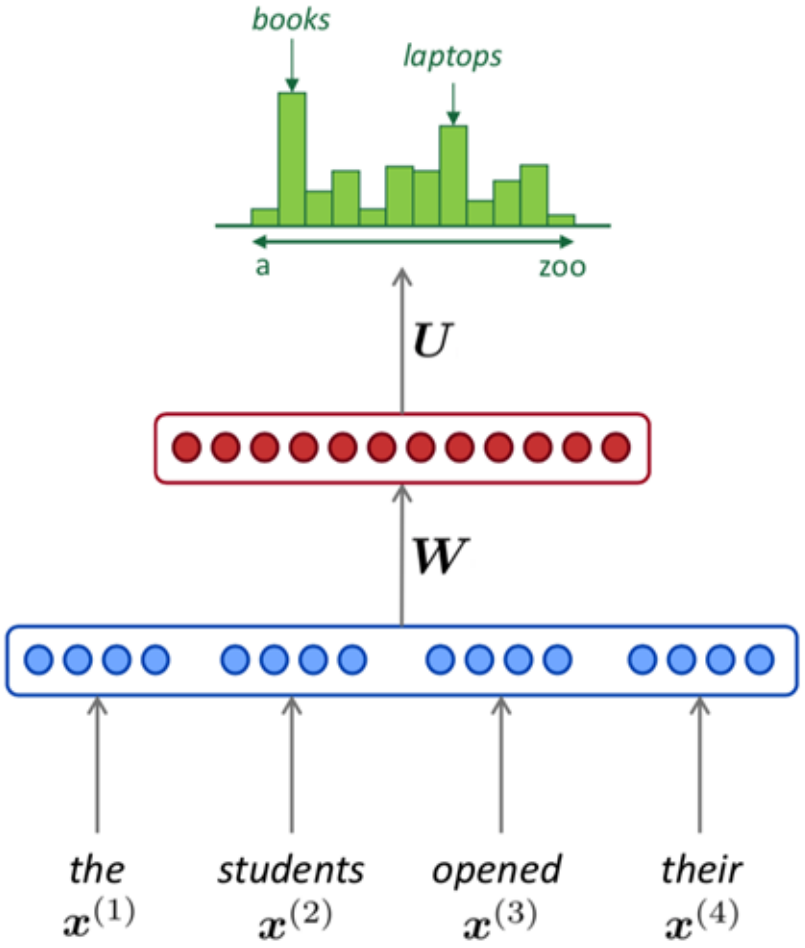
$$h = f(We + b_1)$$

- concatenated word embeddings

$$e = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

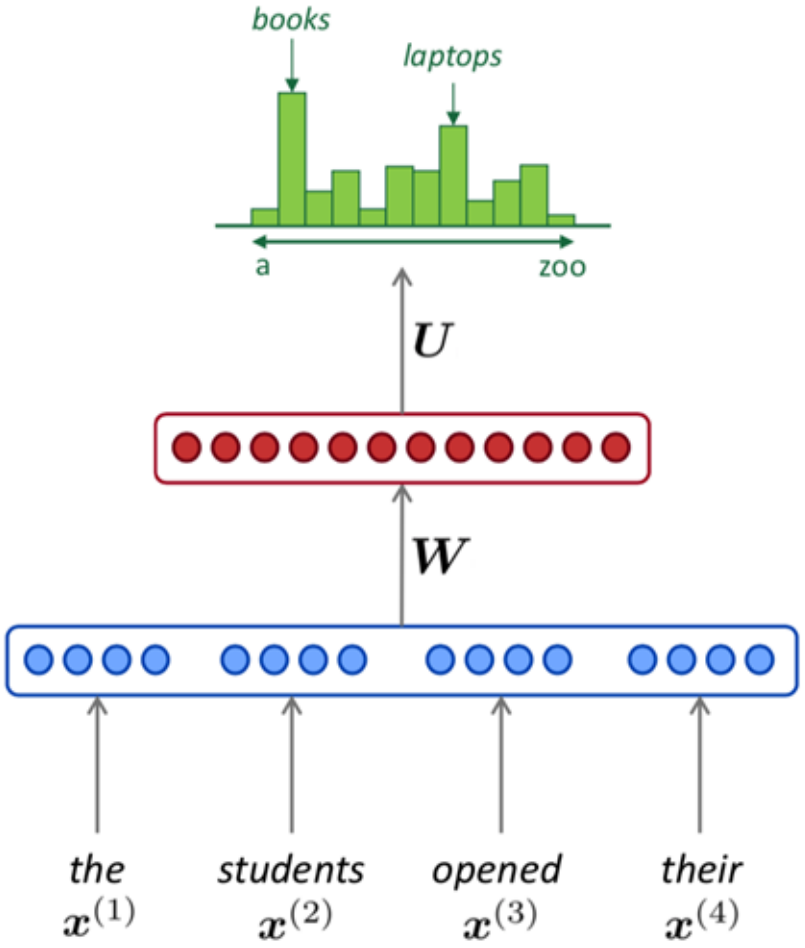
- words/one-hot vectors:

$$x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}$$



A fixed-window neural language model

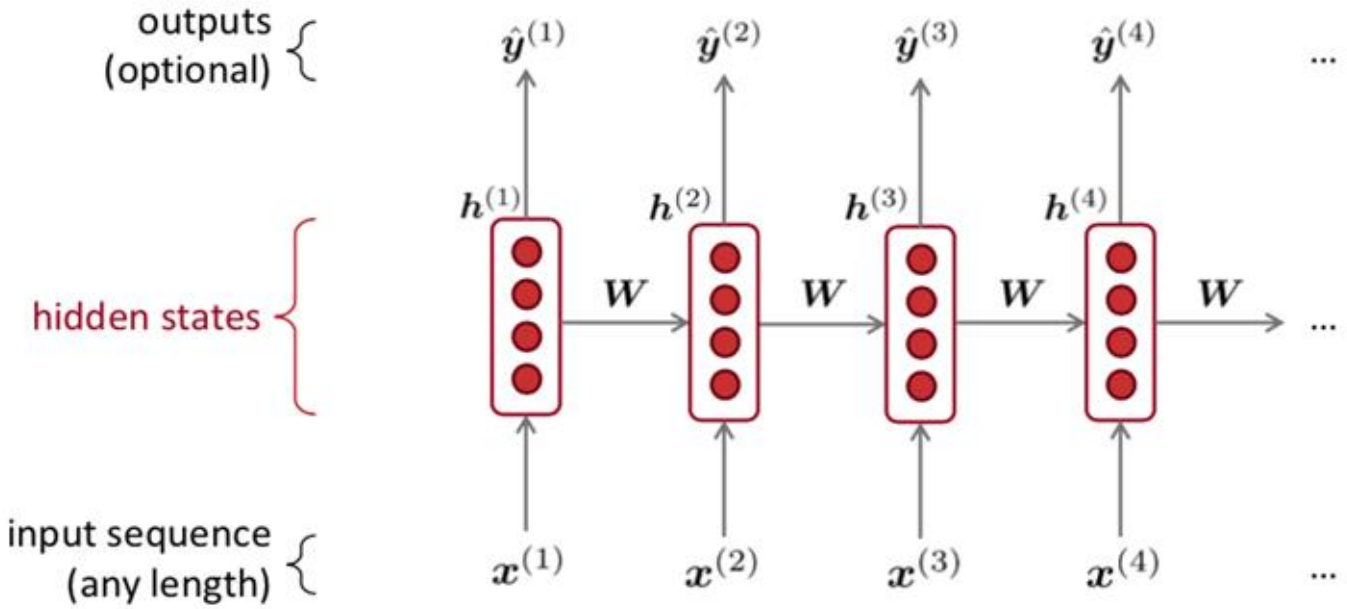
- Improvements over n-gram LM:
  - No sparsity problem
  - Model size is O(n) not O(exp(n))
- Remaining problems:
  - Fixed window is too small
  - Enlarging window enlarges W
  - Window can never be large enough!
  - Each x(i) uses different rows of W. We don't share weights across window.



We need a neural architecture that can process any length input

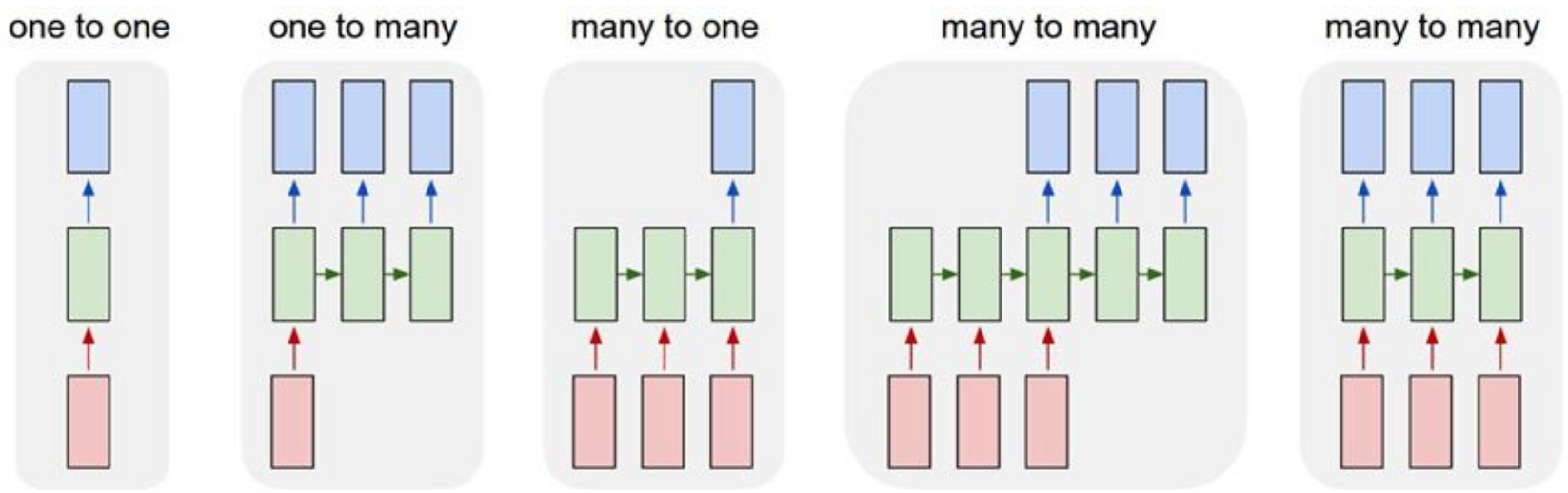
Recurrent Neural Networks (RNN)
A family of neural architectures

- Core idea:
  - Hidden state h_t = “summary of prefix”
  - Apply the same weights W repeatedly



RNN = a function that compresses everything seen so far into a vector.

Recurrent Neural Networks (RNN)
A family of neural architectures



An RNN language model

- output distribution

$$\hat{y}^{(t)} = \text{softmax}(Uh^{(t)} + b_2) \in R^{|V|}$$

- hidden layer

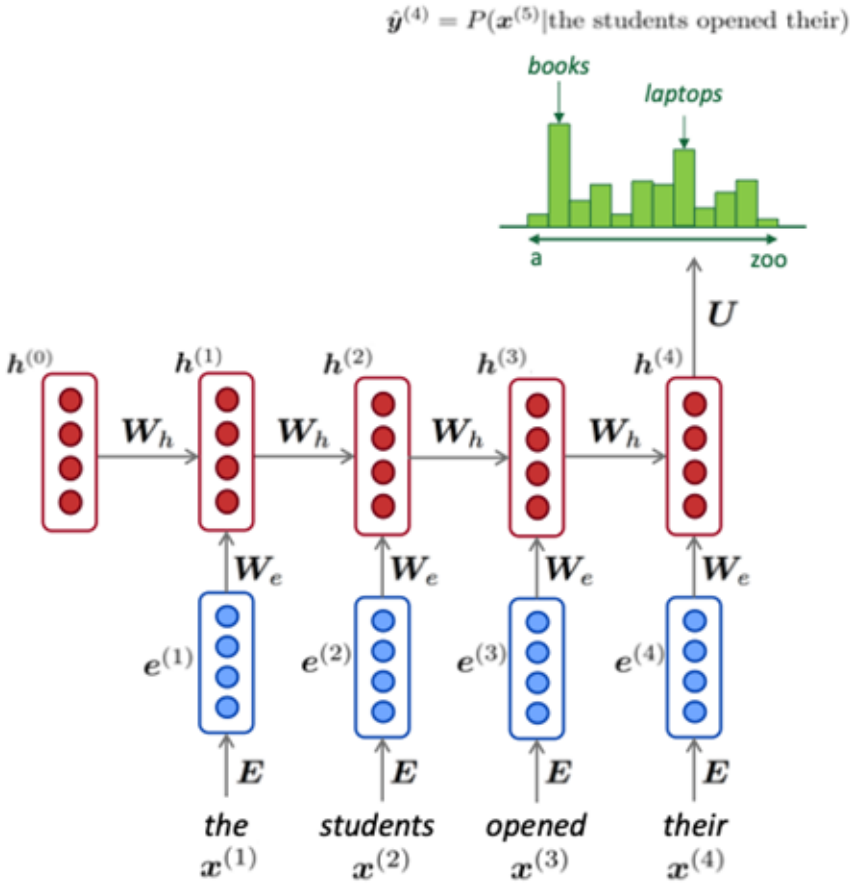
$$h^{(t)} = \tanh(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

- word embeddings:

$$e^{(t)} = Ex^{(t)}$$

- words/one-hot vectors:

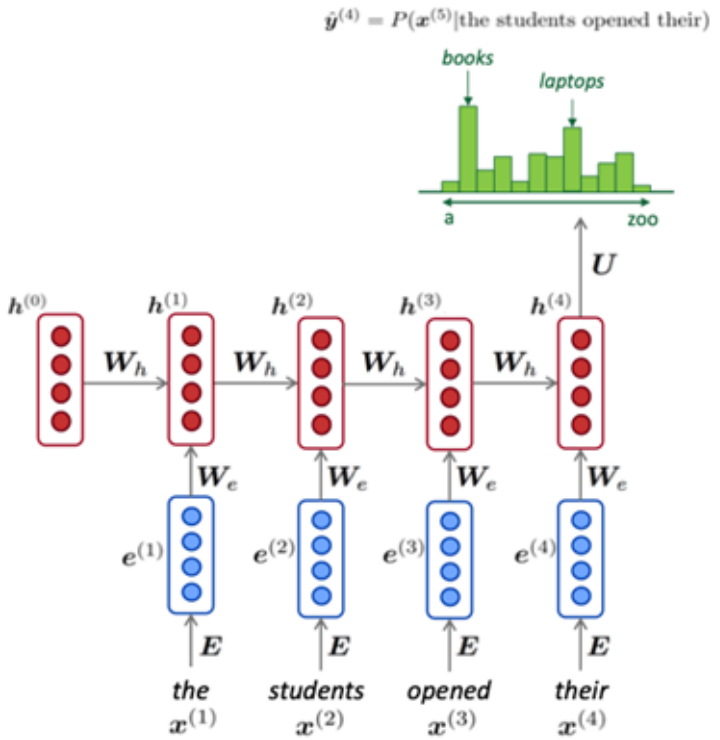
$$x^{(t)} \in R^{|V|}$$



Note: this input sequence could be much longer

An RNN language model

- Advantages:
  - Can process any length input
  - Model size doesn't increase for longer input
  - Computation for step t can use information from many steps back (in theory)
  - Weights are shared across time stamps: representation are shared
- Disadvantages:
  - recurrent computation is slow
  - in practice, difficult to access information from multiple steps back



Training an RNN language model

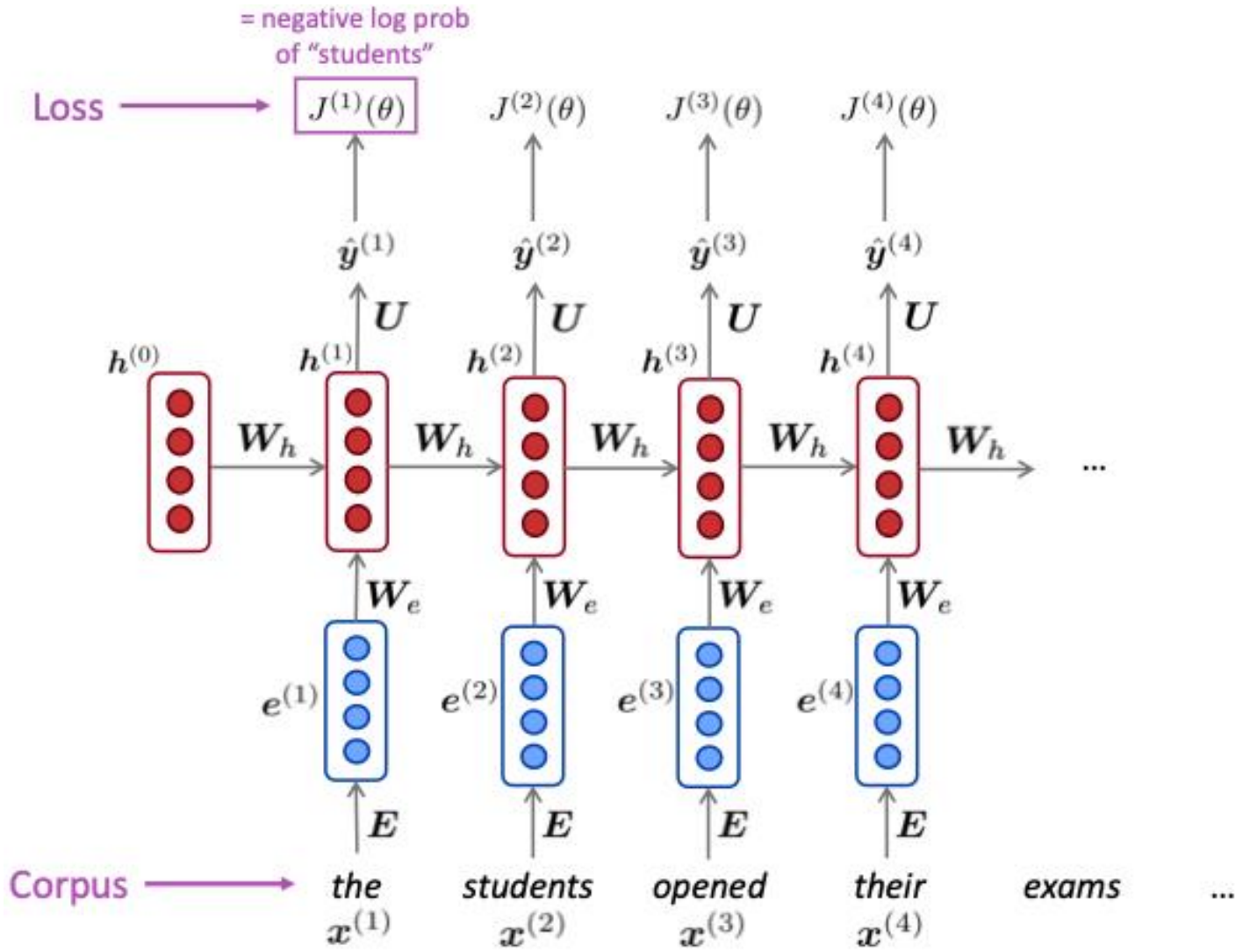
- Get a big corpus of text which is a sequence of words $x^{(1)}, x^{(2)}, \ldots, x^{(t)}$
- Feed into RNN-LM; compute output distribution $\hat{y}^{(t)}$ for every step t.
- Loss function on step t is usual cross-entropy between out predicted probability $\hat{y}^{(t)}$ and the true next word $y^{(t)} = x^{(t+1)}$:

$$J^{(t)}(\theta) = \mathrm{CE}(y^{(t)}, \hat{y}^{(t)}) = -\sum_{j=1}^{|V|} y_j^{(t)} \log \hat{y}_j^{(t)}$$

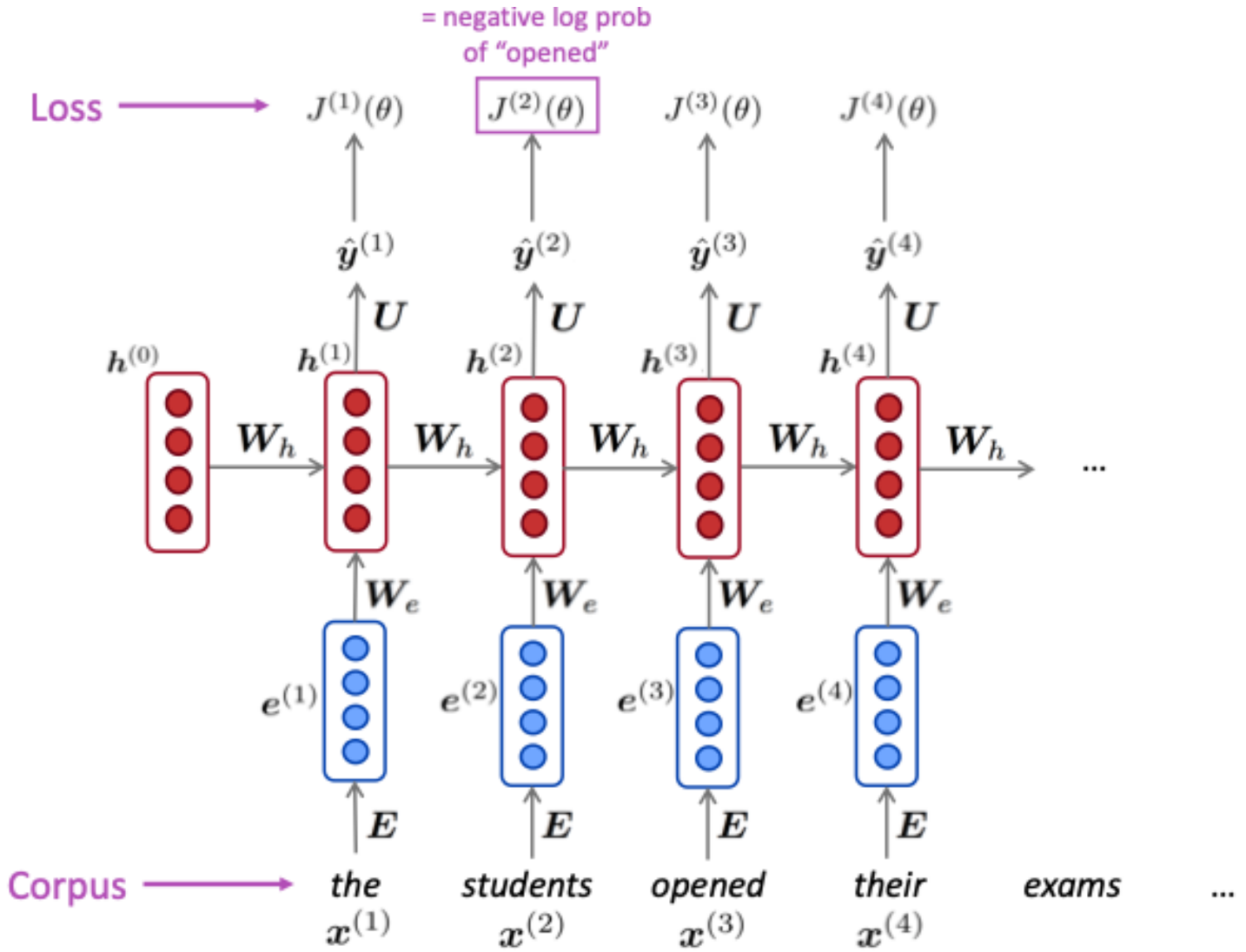
- Average this to get overall loss for entire training set

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J^{(t)}(\theta)$$

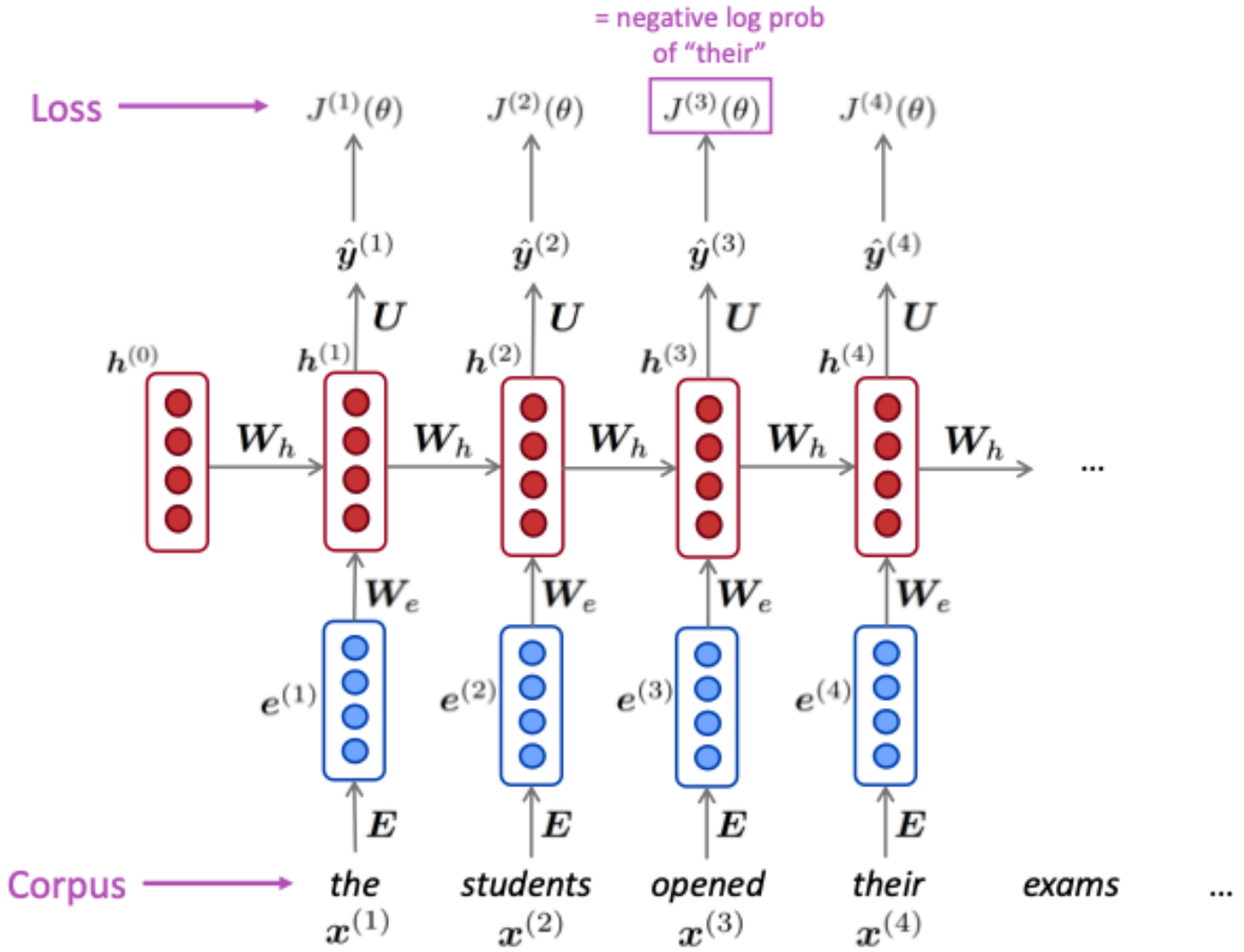
Training an RNN language model



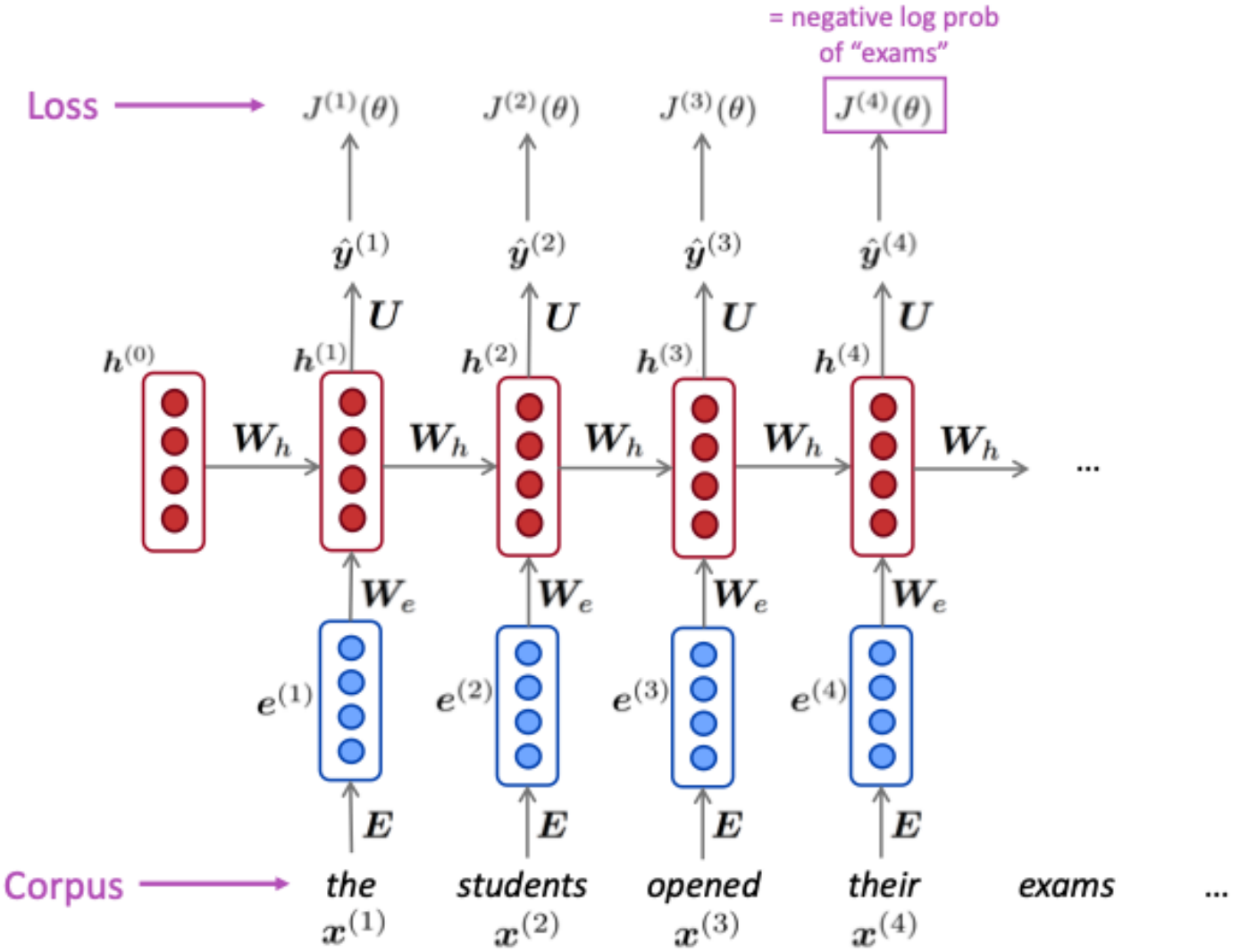
Training an RNN language model



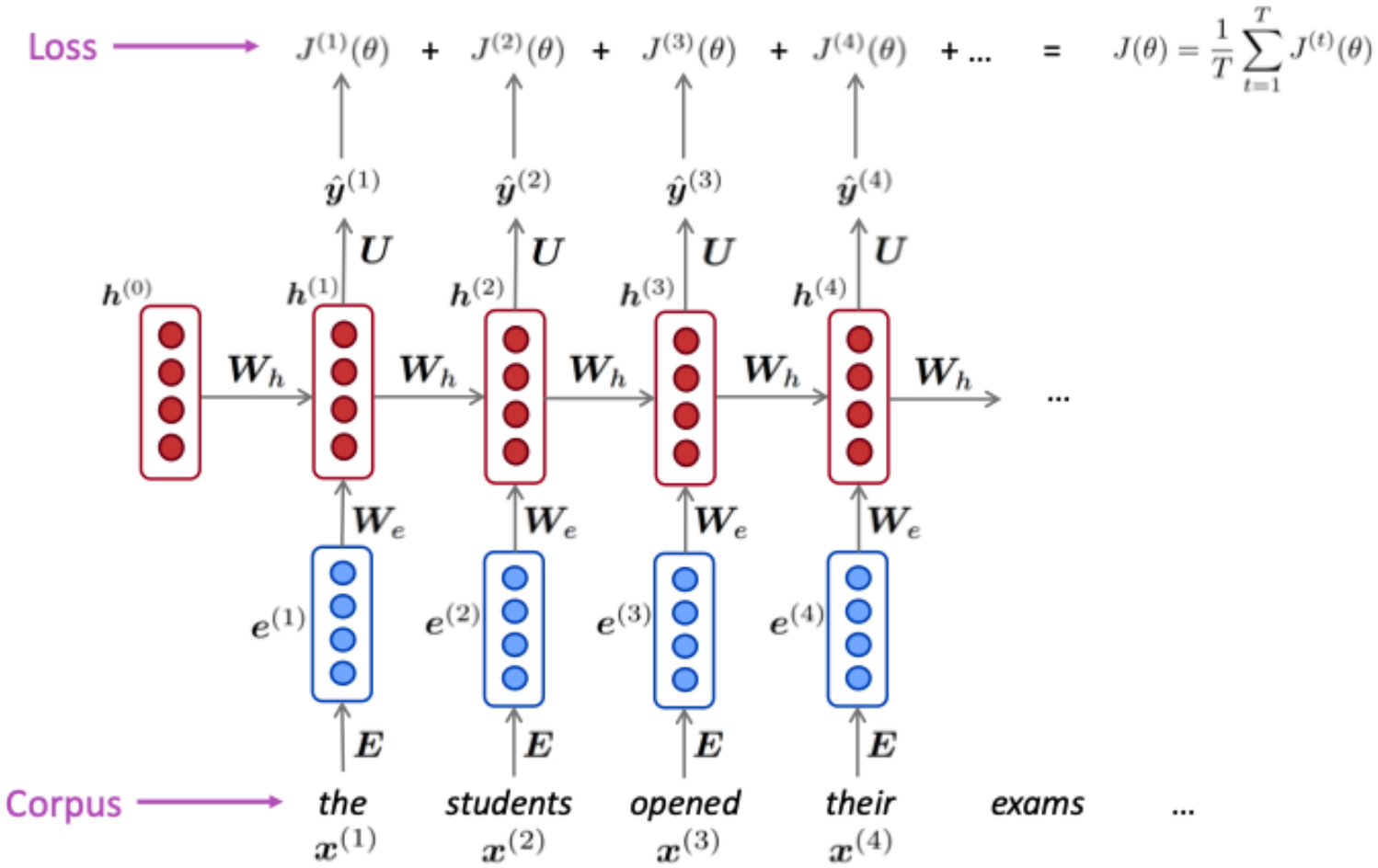
Training an RNN language model



Training an RNN language model



Training an RNN language model



Training an RNN language model

- Computing loss and gradients across entire corpus is too expensive!
- Recall: **Stochastic Gradient Descent** allows us to compute loss and gradient for small set of data and update
- In practice, consider x(1),x(2),…,x(t) as a sentence:

$$J(\theta) = \frac{1}{T}\sum_{t=1}^{T} J^{(t)}(\theta)$$

- Compute loss J(θ) for a sentence, compute gradients and update weights. Repeat.

Backpropagation for RNNs

- What is the derivative of $J(\theta)^{(t)}$ w.r.t the repeated weight matrix W_h?
- Answer:

$$\frac{\partial J^{(t)}}{\partial \mathbf{W}_h} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \mathbf{W}_h}\Big|_{(i)}$$

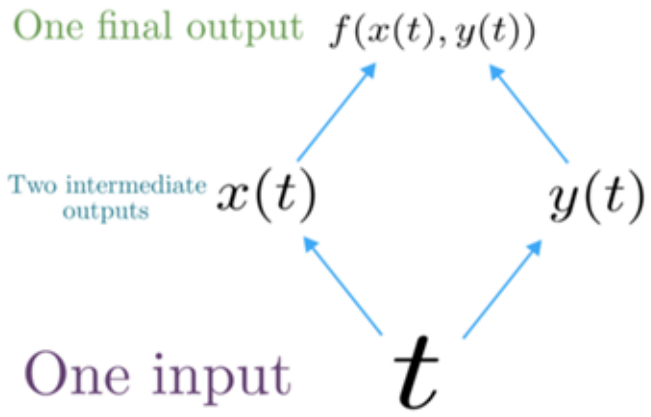
- Backpropagate over time steps i=t,...,0,summing gradients as you go. This algorithm is called "backpropagation through time"

"The gradient w.r.t a repeated weight is the sum of the gradient w.r.t each time it appears"
WHY?

Multivariable chain rule

- Given a multivariable function f(x,y), and two single variable functions x(t) and y(t), here's what the multivariable chain rule says:

$$\underbrace{\frac{d}{dt} f(x(t), y(t))}_{\text{Derivative of composition function}} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}$$



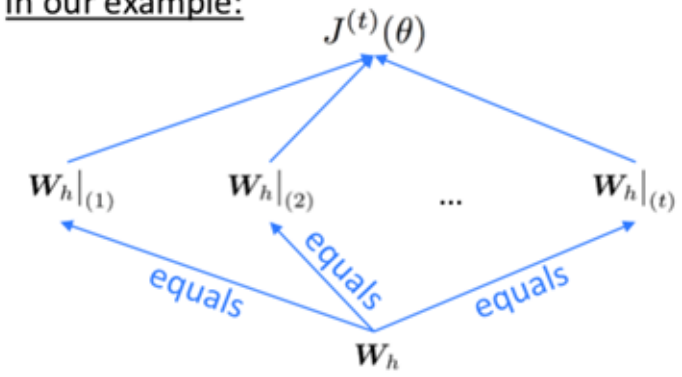
https://www.khanacademy.org/math/multivariable-calculus/multivariable-derivatives/differentiating-vector-valued-functions/a/multivariable-chain-rule-simple-version

Backpropagation for RNNs: proof sketch

- Given a multivariable function f(x,y), and two single variable functions x(t) and y(t), here's what the multivariable chain rule says:

$$\underbrace{\frac{d}{dt} f(x(t), y(t))}_{\text{Derivative of composition function}} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}$$

In our example:

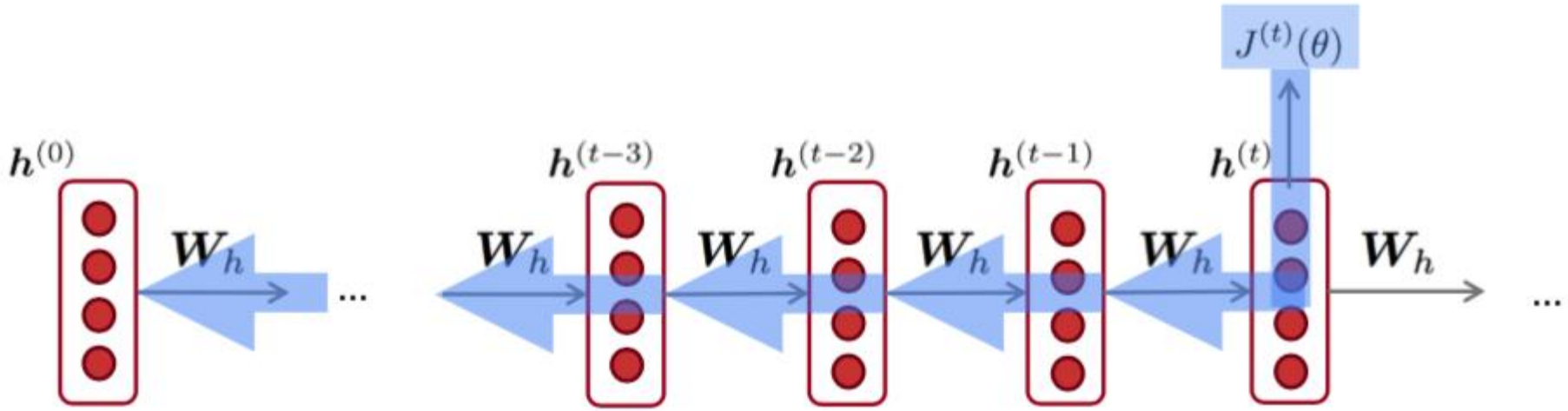


Apply the multivariable chain rule:

$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial W_h}\Big|_{(i)} \frac{\partial W_h|_{(i)}}{\partial W_h} \quad (= 1)$$

$$= \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial W_h}\Big|_{(i)}$$

Backpropagation for RNNs



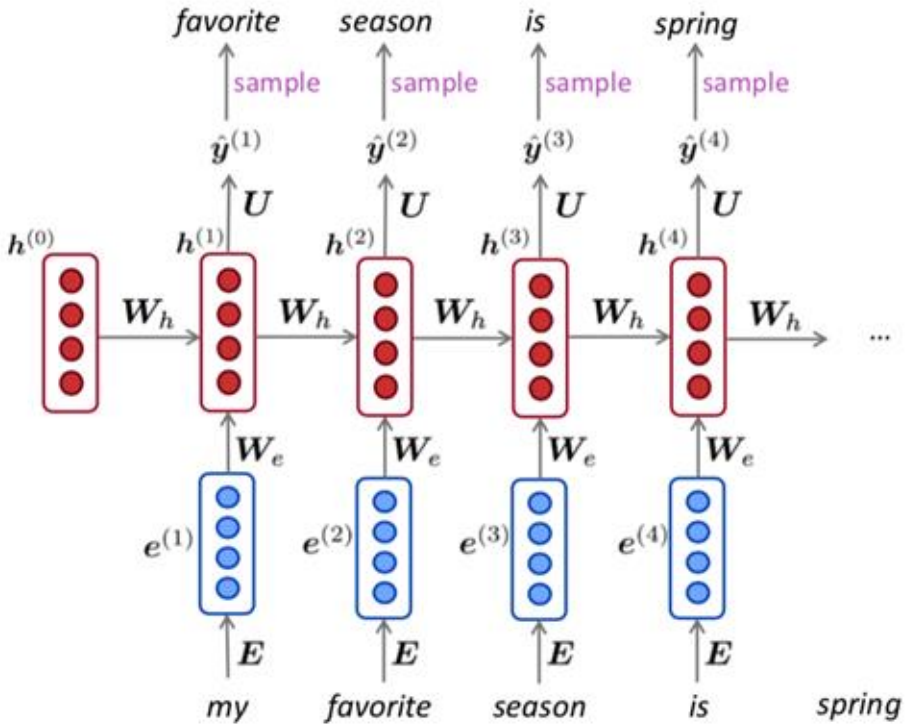
$$\frac{\partial J^{(t)}}{\partial \mathbf{W}_h} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \mathbf{W}_h}\Big|_{(i)}$$

Q: how do we calculate this?

Answer: backpropagate over time steps i = t, … ,0, summing gradients as you go.
This is called “backpropagation through time”

Generating text with an RNN language model

- Just like a n-gram LM, we can use a RNN LM to generate text by repeated sampling. Samplied output is next step's input.



Generating text with an RNN language model

RNN-LM trained on Obama speeches :

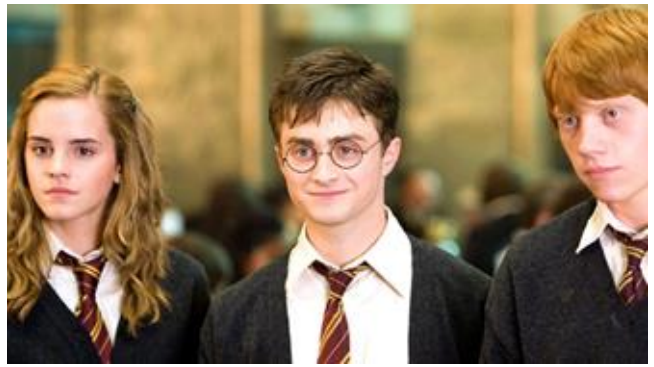
The United States will step up to the cost of a new challenges of the American people that will share the fact that we created the problem. They were attacked and so that they have to say that all the task of the final days of war that I will not be able to get this done. …

https://medium.com/@samim/obama-rnn-machine-generated-political-speeches-c8abd18a2ea0

Generating text with an RNN language model

RNN-LM trained on *Harry Potter*:

"Sorry", Harry shouted, panicking -- "I'll lease those brooms in London, are they?"
"No idea", said Nearly Headless Nick, casting low close by Cedric, carrying the last bit of treacle of Charms, from Harry's shoulder, and to answer him the common room perched upon it, four arms held a shining knob from when the spider hadn't felt it seemed, He reached the teams too.



https://medium.com/deep-writing/harry-potter-written-by-artificial-intelligence-8a9431803da6

Generating text with an RNN language model

RNN-LM trained on recipes:

```
MMMMM----- Recipe via Meal-Master (tm) v8.05

     Title: CARAMEL CORN GARLIC BEEF
 Categories: Soups, Desserts
     Yield: 10 Servings

     2 tb Parmesan cheese, ground
   1/4 ts Ground cloves
          -- diced
     1 ts Cayenne pepper

 Cook it with the batter. Set aside to cool. Remove the peanut oil in a small saucepan and
 pour into the margarine until they are soft. Stir in a
 a mixer (dough). Add the chestnuts, beaten egg whites, oil, and salt
 and brown sugar and sugar; stir onto the boqtly brown it.

 The recipe from an oiled by fried and can. Beans, by Judil Cookbook, Source: Pintore,
 October, by Chocolates, Breammons of Jozen, Empt.com
```

https://gist.github.com/nylki/1efbaa36635956d35bcc

Generating text with an RNN language model

RNN-LM trained on paint color names:

https://aiweirdness.com/post/160776374467/new-paint-colors-invented-by-neural-network

Evaluating language models

- The standard evaluation metric for language models is perplexity

$$\text{perplexity} = \prod_{t=1}^{T} \left(\frac{1}{P_{LM}(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \ldots, \mathbf{x}^{(1)})} \right)^{1/T}$$

Normalized by number of words

Inverse probability of corpus, according to LM

- This is equal to the exponential of the cross-entropy loss J(θ)

$$= \prod_{t=1}^{T} \left(\frac{1}{\hat{y}_{x_{t+1}}^{(t+1)}} \right)^{1/T} = \exp\left(\frac{1}{T} \sum_{t=1}^{T} -\log \hat{y}_{x_{t+1}}^{(t)} \right) = \exp(J(\theta))$$

Lower perplexity is better

RNNs have greatly improved perplexity

n-gram model

Increasingly complex RNNs

Model	Perplexity
Interpolated Kneser-Ney 5-gram (Chelba et al., 2013)	67.6
RNN-1024 + MaxEnt 9-gram (Chelba et al., 2013)	51.3
RNN-2048 + BlackOut sampling (Ji et al., 2015)	68.3
Sparse Non-negative Matrix factorization (Shazeer et al., 2015)	52.9
LSTM-2048 (Jozefowicz et al., 2016)	43.7
2-layer LSTM-8192 (Jozefowicz et al., 2016)	30
Ours small (LSTM-2048)	43.9
Ours large (2-layer LSTM-2048)	39.8

https://research.fb.com/building-an-efficient-neural-language-model-over-a-billion-words/

Why should we care about LM?

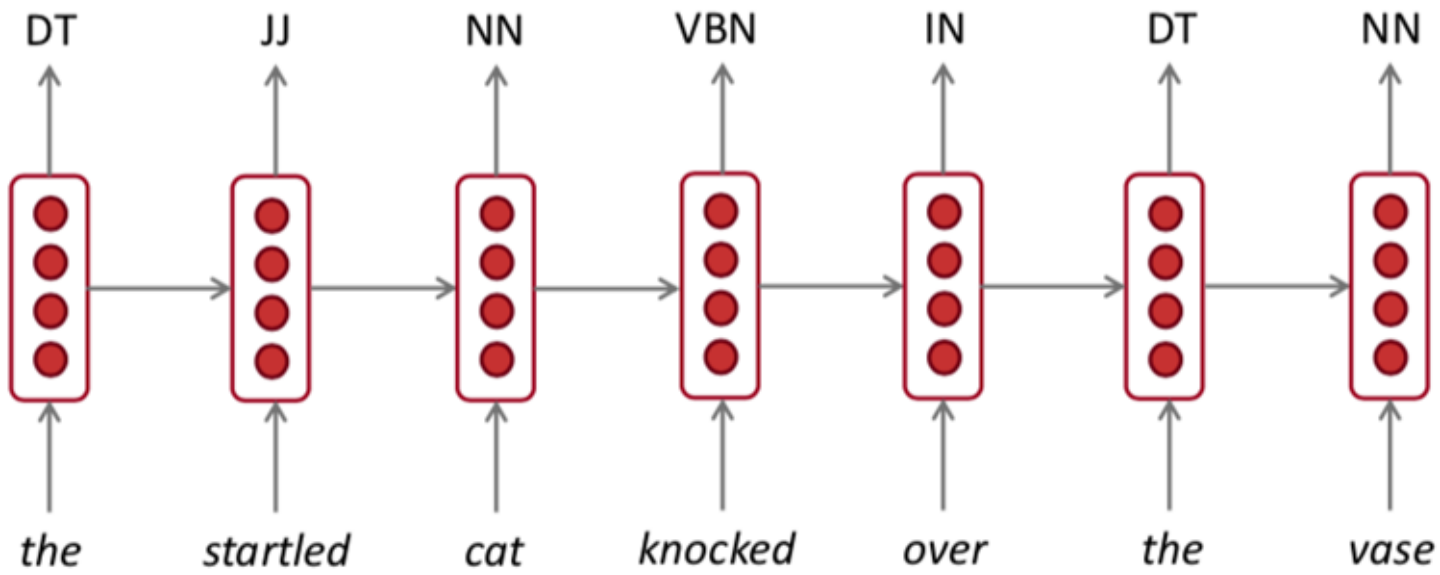
- Language Modeling is a benchmark task that helps us measure our progress on understanding language
- Language Modeling is a subcomponent of many NLP tasks, especially those involving generating text or estimating the probability of text:
  - Predictive typing
  - Speech recognition
  - Handwriting recognition
  - Spelling/grammar correction
  - Authorship identification
  - Machine translation
  - Summarization
  - Dialogue

Summary

- Language Model: A system that predicts the next word
- Recurrent Neural Network: a family of neural networks that
  - Take sequential input of any length
  - Apply the same weights on each step
  - Can optionally produce output on each step
- RNNs ⇏ Language Model
- We've shown that RNNs are a great way to build a LM
- but RNNs are useful for much more!

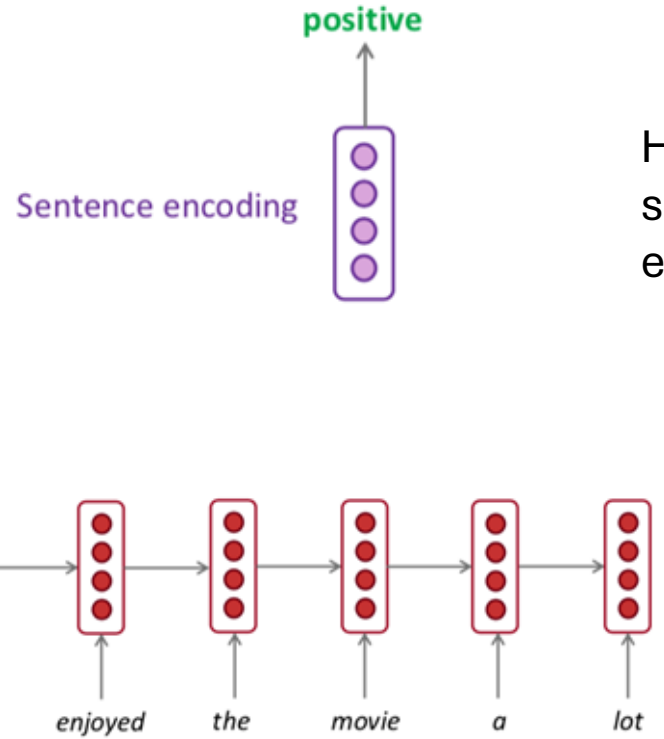
RNNs can be used for tagging

part-of-speech tagging, named entity recognition



RNNs can be used for sentence classification

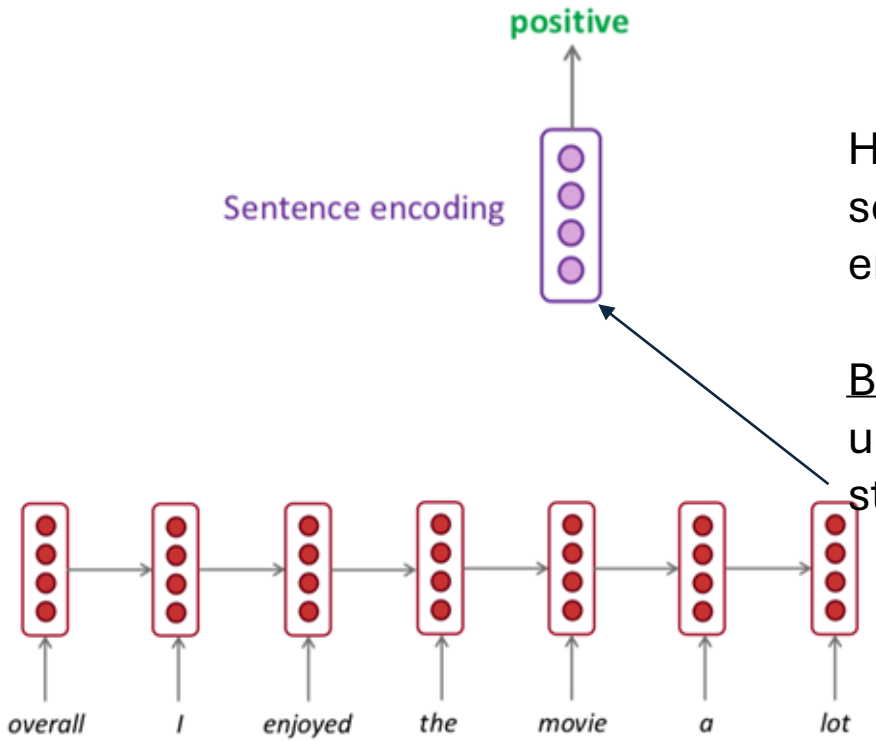
sentiment classification



How to compute sentence encoding?

RNNs can be used for sentence classification

sentiment classification

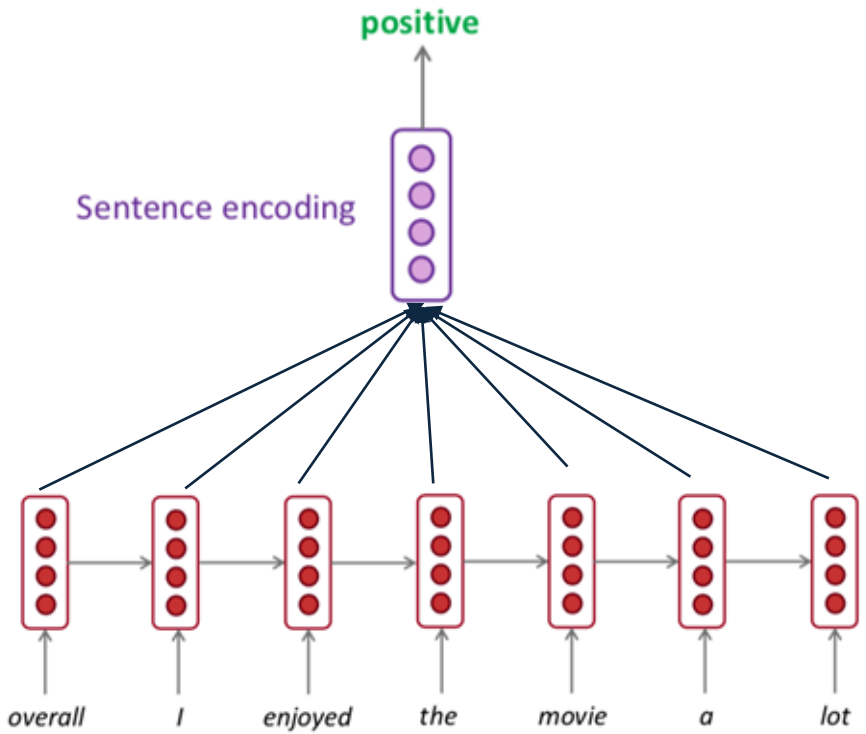


How to compute sentence encoding?

<u>Basic way:</u>
use final hidden state

RNNs can be used for sentence classification

sentiment classification

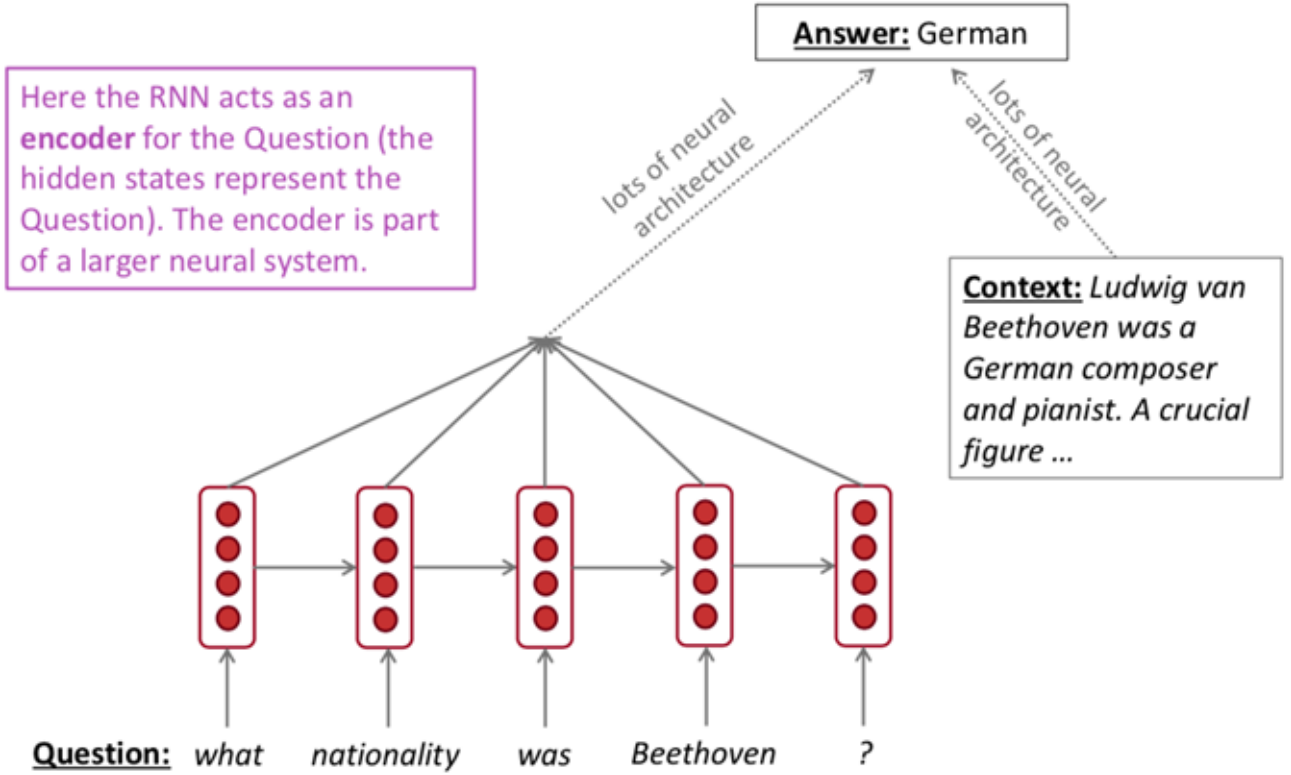


How to compute sentence encoding?

<u>Usually better:</u> take element-wise max or mean of all hidden states

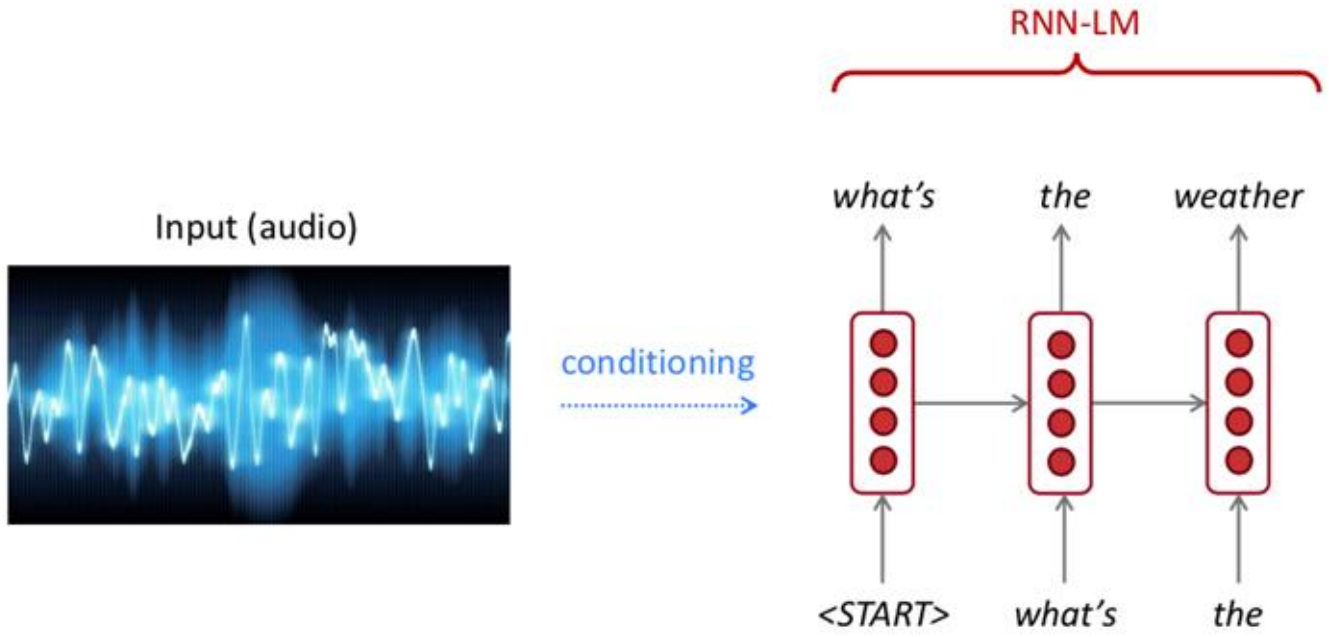
RNNs can be used for encoder module

question answering, machine translation, many other tasks!



RNNs can be used to generate text

speech recognition, machine translation, summarization



This is an example of a conditional language model. See Machine Translation in much more detail later.

A note on terminology

- RNN described in this lecture = “vanilla RNN” 
- Next lecture: learn about other RNN flavors
  - like GRU  and LSTM  and multi-layer RNNs 
- By the end of the course, you will understand phrases like
  - “stacked bidirectional LSTM with residual connections and self-attention”

Readings

- CH 9, 13 SLP; CH 14 NNLP
- A Survey of Large Language Models: https://arxiv.org/pdf/2303.18223.pdf
- N-gram language models: https://web.stanford.edu/~jurafsky/slp3/3.pdf

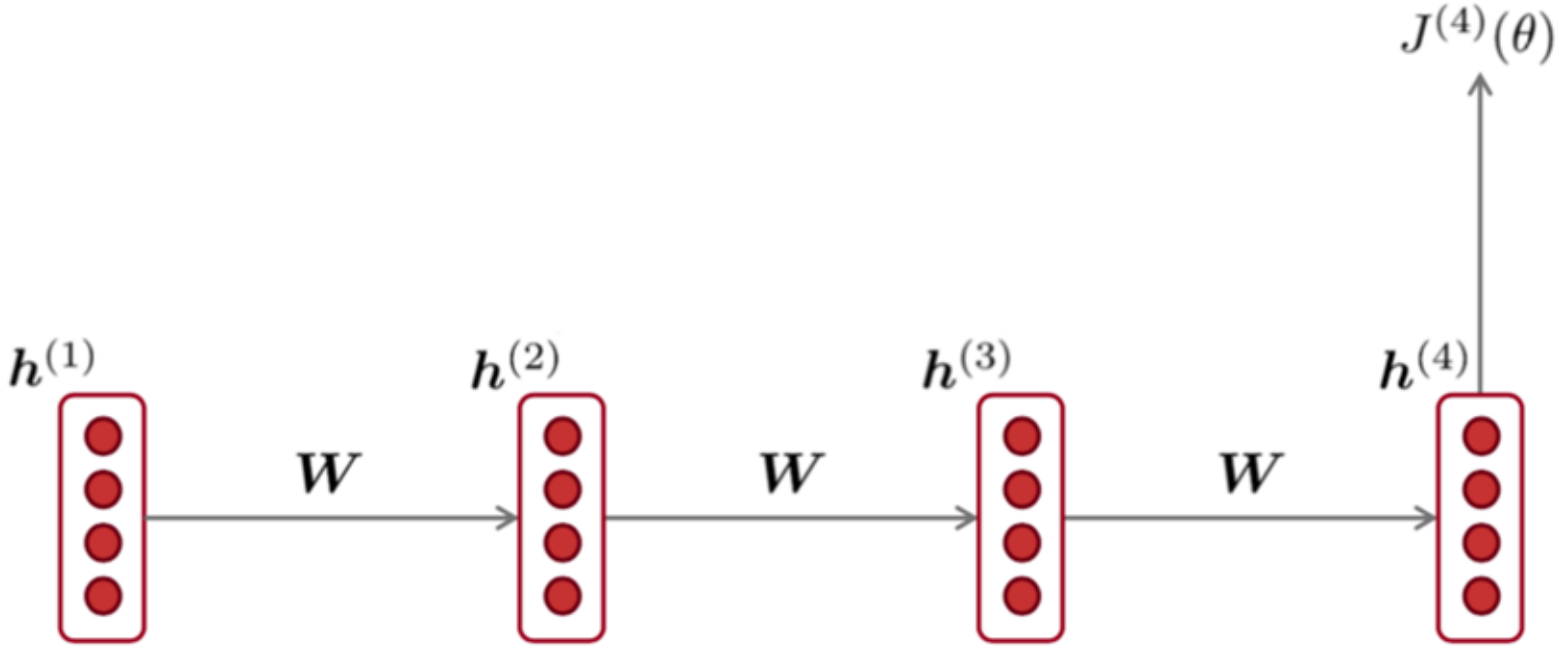
- Recurrent neural networks cheatsheet: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

- Implementing RNN from scratch: https://github.com/pangolulu/rnn-from-scratch

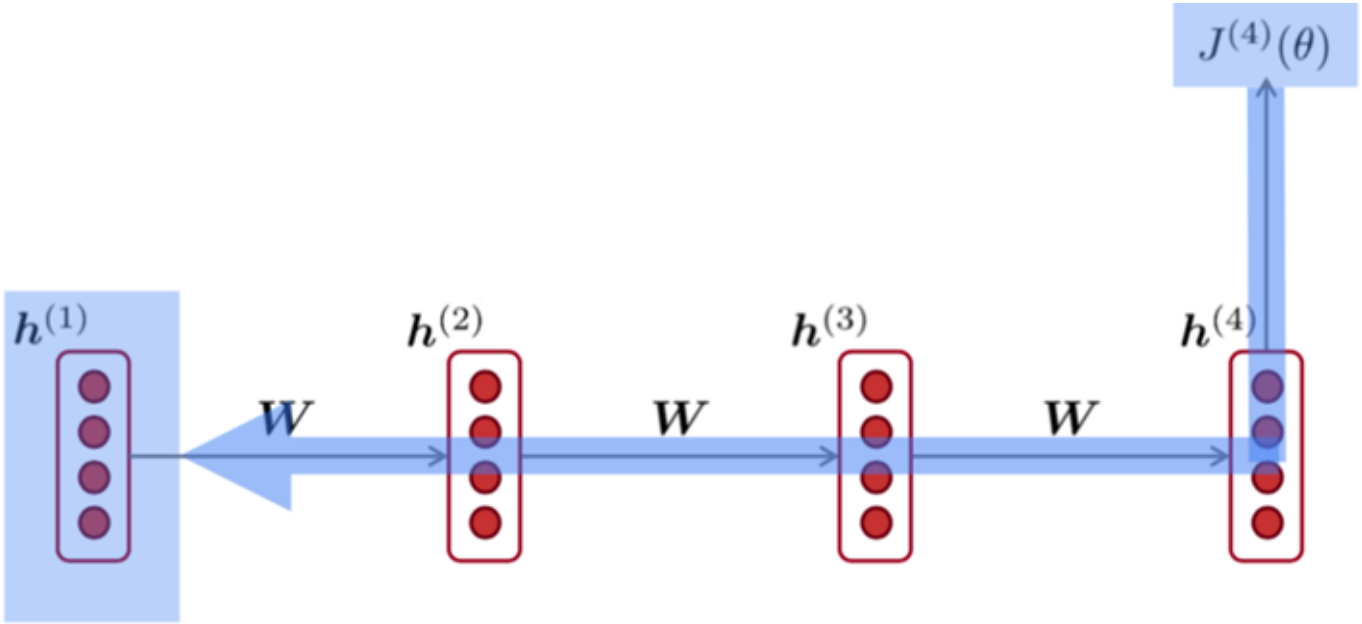
Part 2

Vanishing Gradient -> LSTM

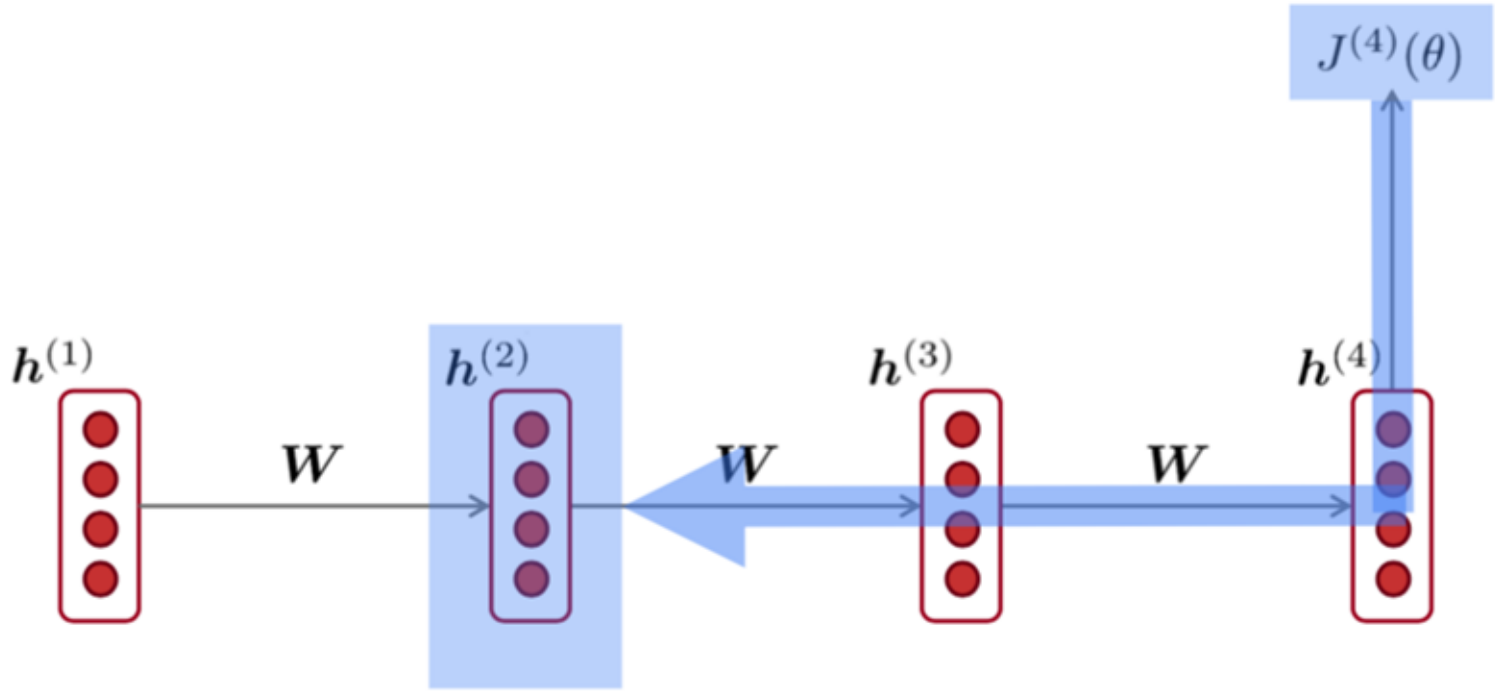
Vanishing gradient intuition



Vanishing gradient intuition



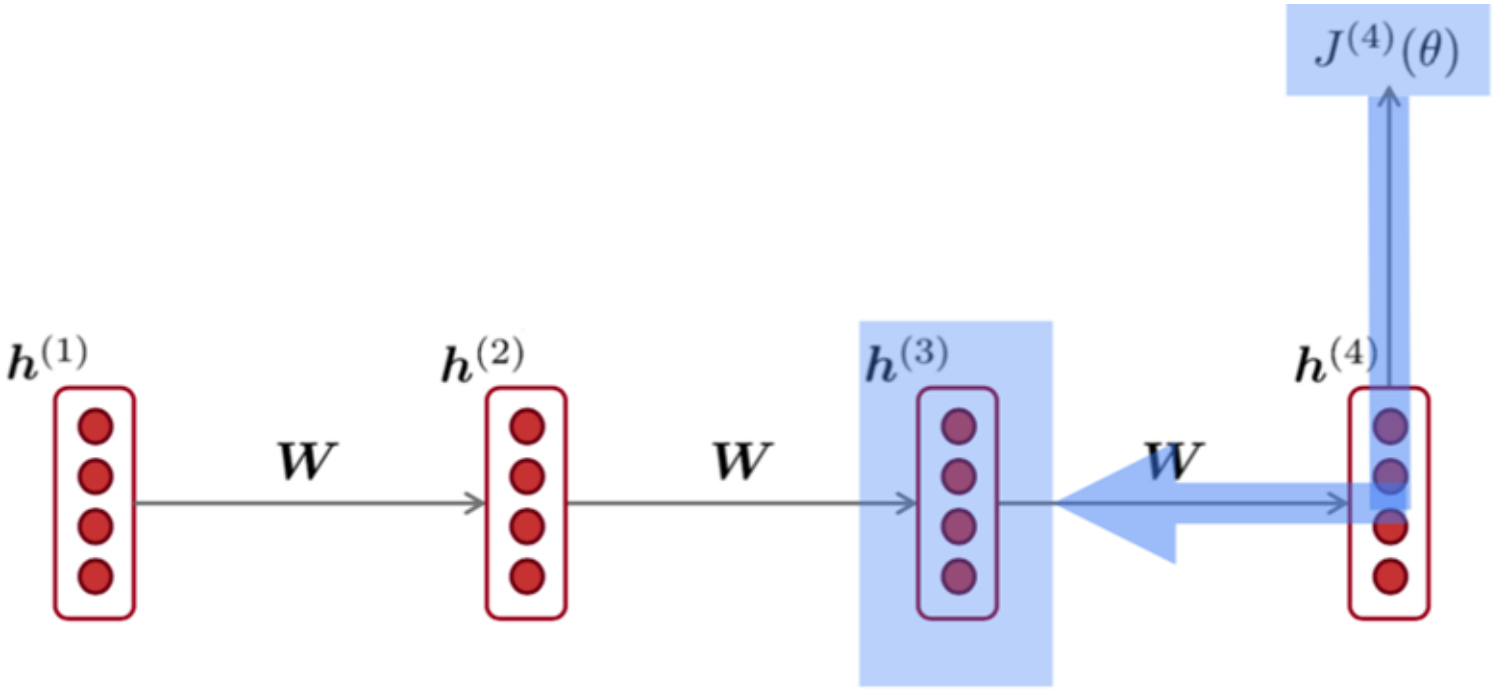
Vanishing gradient intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial J^{(4)}}{\partial h^{(2)}}$$

chain rule!

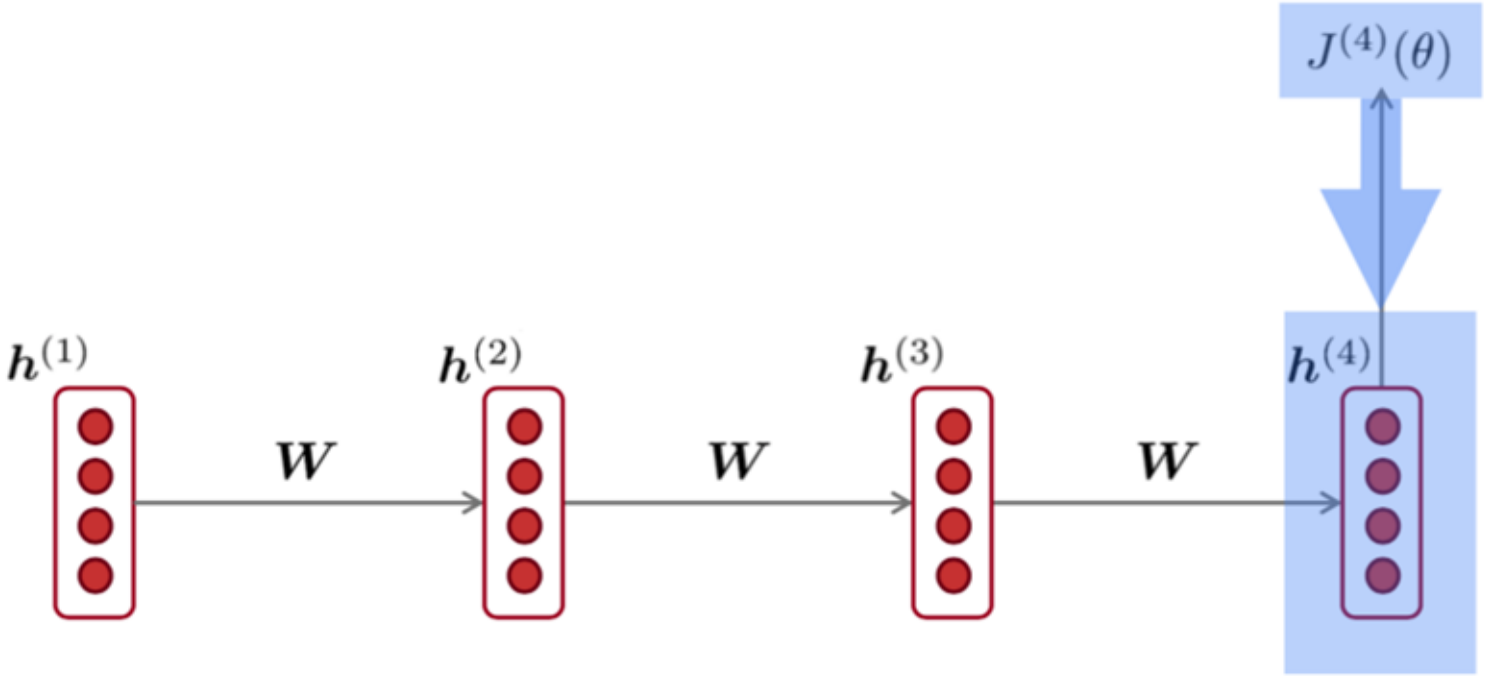
Vanishing gradient intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial J^{(4)}}{\partial h^{(3)}}$$

chain rule!

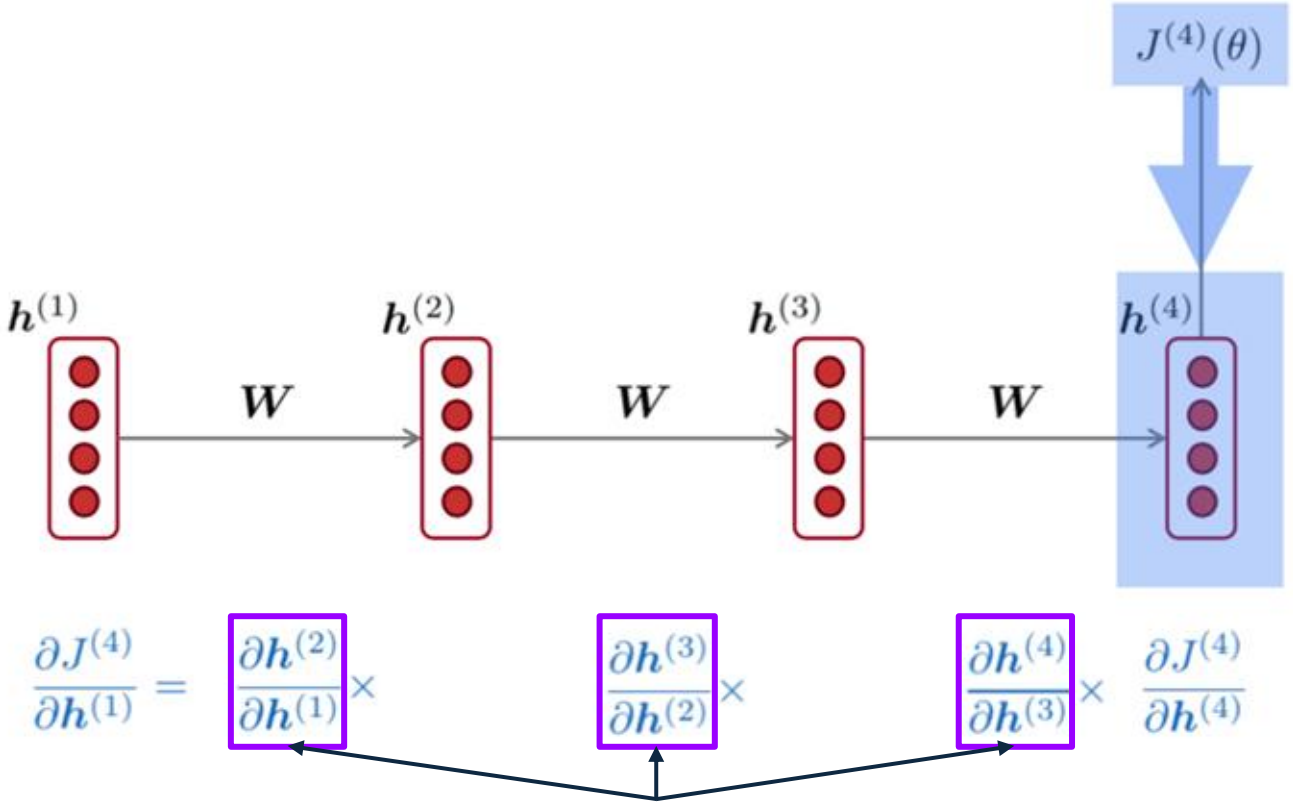
Vanishing gradient intuition



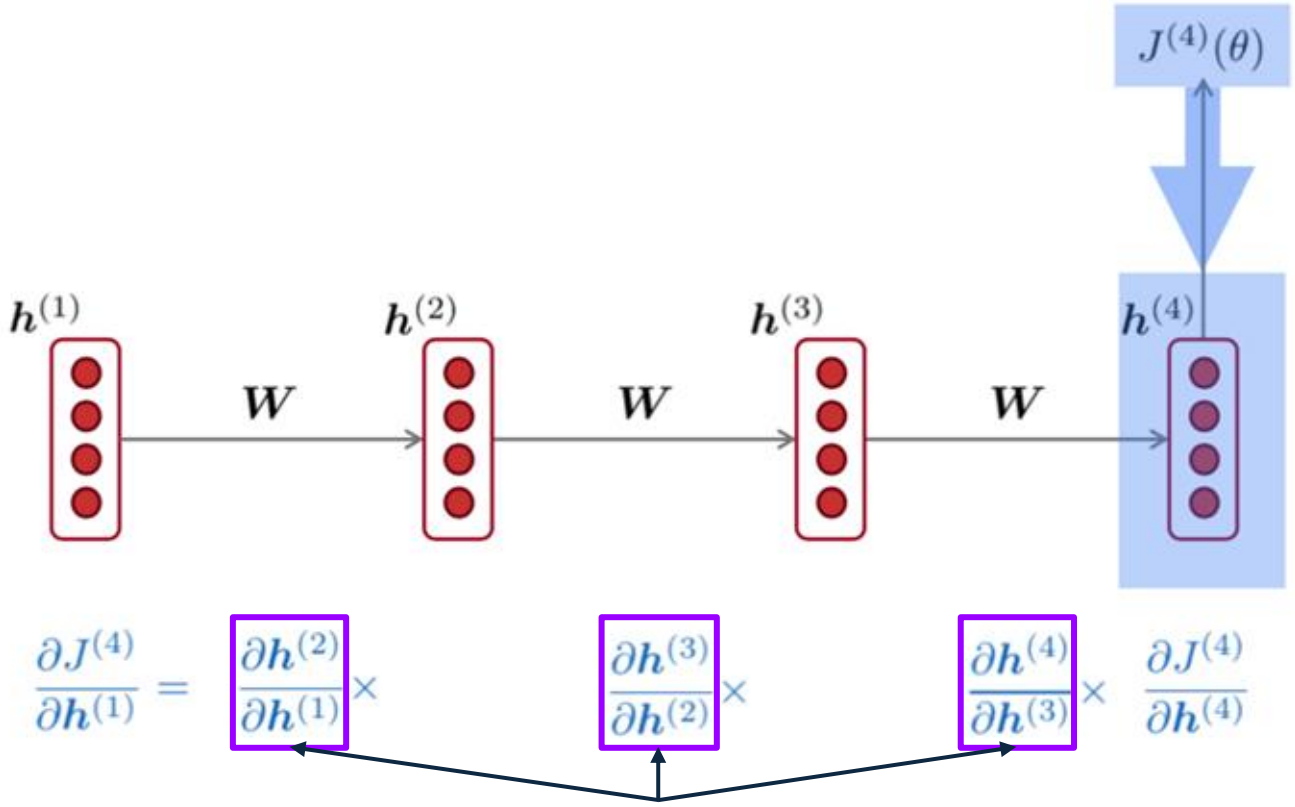
$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

chain rule!

Vanishing gradient intuition



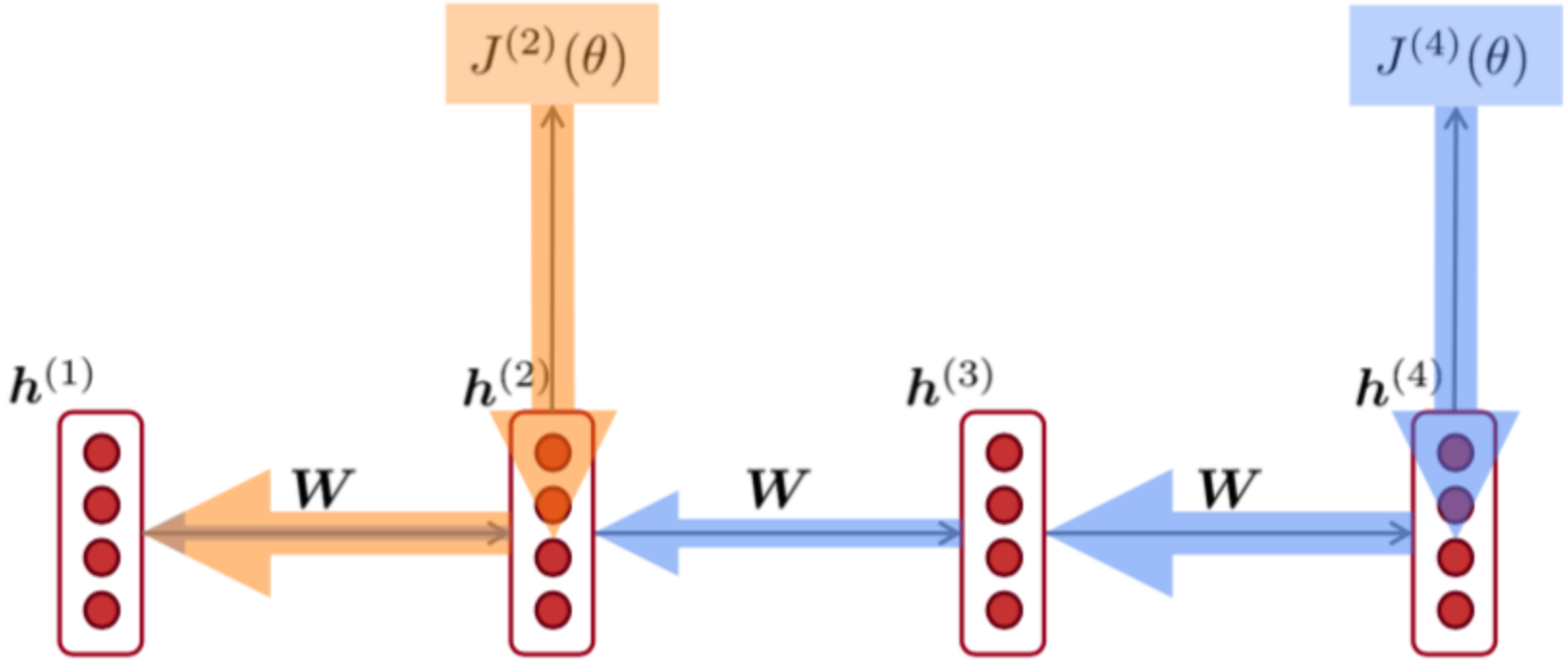
Vanishing gradient intuition



What if these are small?

Vanishing gradient problem: when these are small, the gradient signal gets smaller and smaller as it backpropagates further.

Why is vanishing gradient a problem?



Gradient signal from far away is lost because it's much smaller than gradient signal from close-by.

So model weights are only updated w.r.t near effects, not long-term effects

Why is vanishing gradient a problem?

- Another explanation: gradient can be viewed as a measure of the effect of the past on the future
- If the gradient becomes vanishingly small over longer distances (step t to step t+n), then we can't tell whether:
  - There's no dependency between step t and t+n in the data
  - We have wrong parameters to capture the true dependency between t and t+n

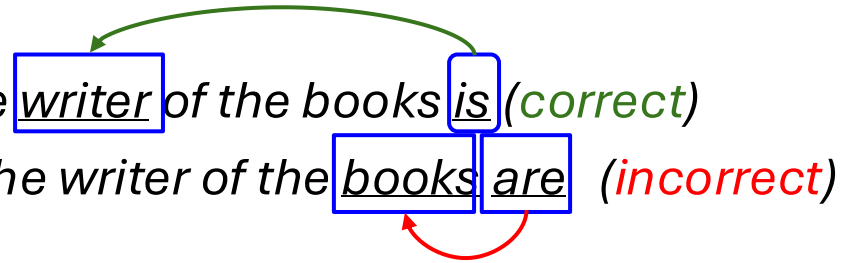
Effect of vanishing gradient on RNN-LM

- LM task: *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her_____*
- To learn from this training example, the RNN-LM needs to model the dependency between "tickets" on the 7th step and the target word "tickets" at the end.

- But if gradient is small, the model can't learn this dependency
  - So the model is unable to predict similar long-distance dependencies at test time

Effect of vanishing gradient on RNN-LM

- LM task: *The writer of the books _____ (is? are?)*
- Correct answer: *The writer of the books is planning a sequel*



- Syntactic recency: *The writer of the books is (correct)*
- Sequential recency: *The writer of the books are (incorrect)*

- Due to vanishing gradient, RNN-LMs are better at learning from sequential recency than syntactic recency, so they make this type of error more often than we'd like [Linzen et al 2016]

Why is exploding gradient a problem?

- If the gradient becomes too big, then the SGD update step becomes too big:

learning rate

$$\theta^{\text{new}} = \theta^{\text{old}} - \eta \nabla_\theta J(\theta)$$

gradient

- This can cause bad updates: we take too large a step and reach a bad parameter configuration (with large loss)
- In the worst case, this will result in Inf or NaN in your network (then you have to restart training from an earlier checkpoint)

Gradient clipping: solution for exploding gradient

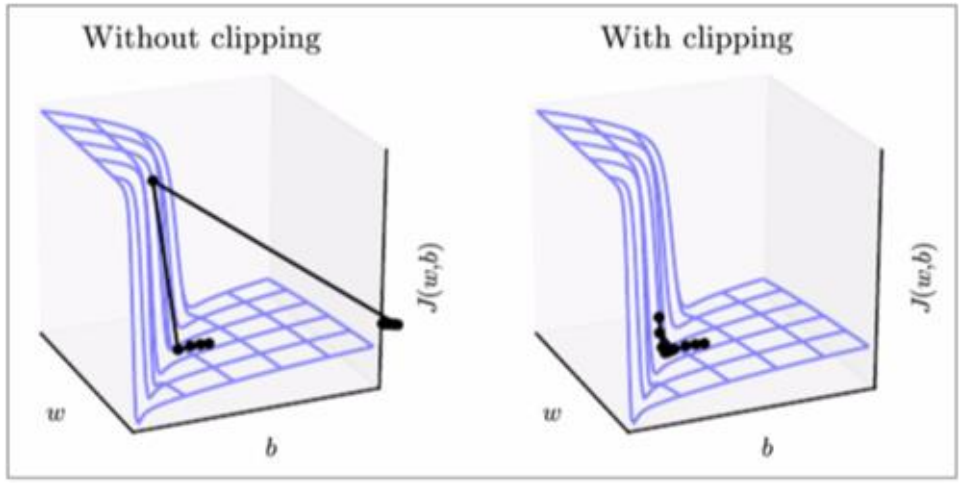
- Gradient clipping: if the norm of the gradient is greater than some threshold, scale it down before applying SGD update

Algorithm 1 Pseudo-code for norm clipping

$\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$
if $\|\hat{\mathbf{g}}\| \geq threshold$ **then**
  $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|}\hat{\mathbf{g}}$
end if

- Intuition: take a step in the same direction, but a smaller step

Gradient clipping: solution for exploding gradient



- This shows the loss surface of a simple RNN (hidden state is a scalar not a vector)
- The "cliff" is dangerous because it has steep gradient
- On the left, gradient descent takes two very big steps due to steep gradient, resulting in climbing the cliff then shooting off to the right (both bad updates)
- On the right, gradient clipping reduces the size of those steps, so the effect is less drastic

How to fix vanishing gradient problem?

- The main problem is that it's too difficult for the RNN to learn to preserve information over many timesteps.

- In a vanilla RNN, the hidden state is constantly being rewritten

$$\mathbf{h}^{(t)} = \sigma\left(W_h \mathbf{h}^{(t-1)} + W_x \mathbf{x}^{(t)} + \mathbf{b}_1\right)$$

- How about a RNN with separate memory?
- →LSTM and GRU

Long Short-Term Memory (LSTM)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.

- On step t, there is a hidden state $h^{(t)}$ and a cell state $c^{(t)}$
  - Both are vectors of length n
  - The cell stores long-term information
  - The LSTM can erase, write and read information from the cell

- The selection of which information is erased/written/read is controlled by three corresponding gates
  - The gates are also vectors length n
  - On each timestep, each element of the gates can be open (1), closed (0), or somewhere in-between.
  - The gates are dynamic: their value is computed based on the current context

Long Short-Term Memory (LSTM)

We have a sequence of inputs x(t), and we will compute a sequence of hidden states h(t), and cell states c(t). on time step t:

Forget gate: controls what is kept vs forgotten, from previous cell state

$$\mathbf{f}^{(t)} = \sigma(W_f \mathbf{h}^{(t-1)} + U_f \mathbf{x}^{(t)} + \mathbf{b}_f)$$

Input gate: controls what parts of the new cell content are written to cell

$$\mathbf{i}^{(t)} = \sigma(W_i \mathbf{h}^{(t-1)} + U_i \mathbf{x}^{(t)} + \mathbf{b}_i)$$

Output gate: controls what parts of cell are output to hidden state

$$\mathbf{o}^{(t)} = \sigma(W_o \mathbf{h}^{(t-1)} + U_o \mathbf{x}^{(t)} + \mathbf{b}_o)$$

New cell content: this is the new content to be written to the cell

$$\tilde{\mathbf{c}}^{(t)} = \tanh(W_c \mathbf{h}^{(t-1)} + U_c \mathbf{x}^{(t)} + \mathbf{b}_c)$$

Cell content: erase ("forget") some content from last cell state, and write ("input") some new cell content

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \circ \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \circ \tilde{\mathbf{c}}^{(t)}$$

Hidden state: read ("output") some content from the cell

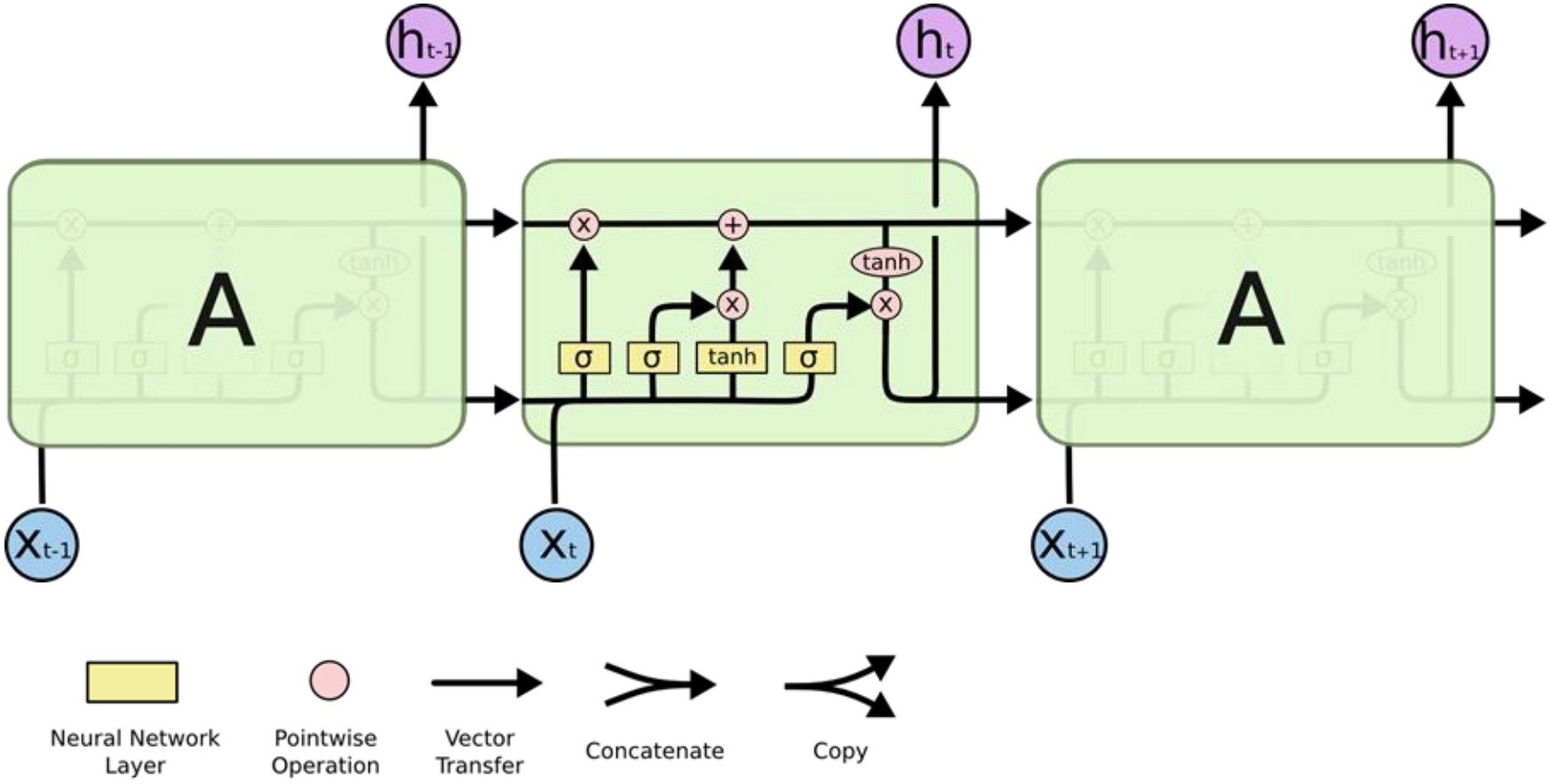
$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \circ \tanh \mathbf{c}^{(t)}$$

Gates are applied using element-wise product

Long Short-Term Memory (LSTM)

controlled information highway

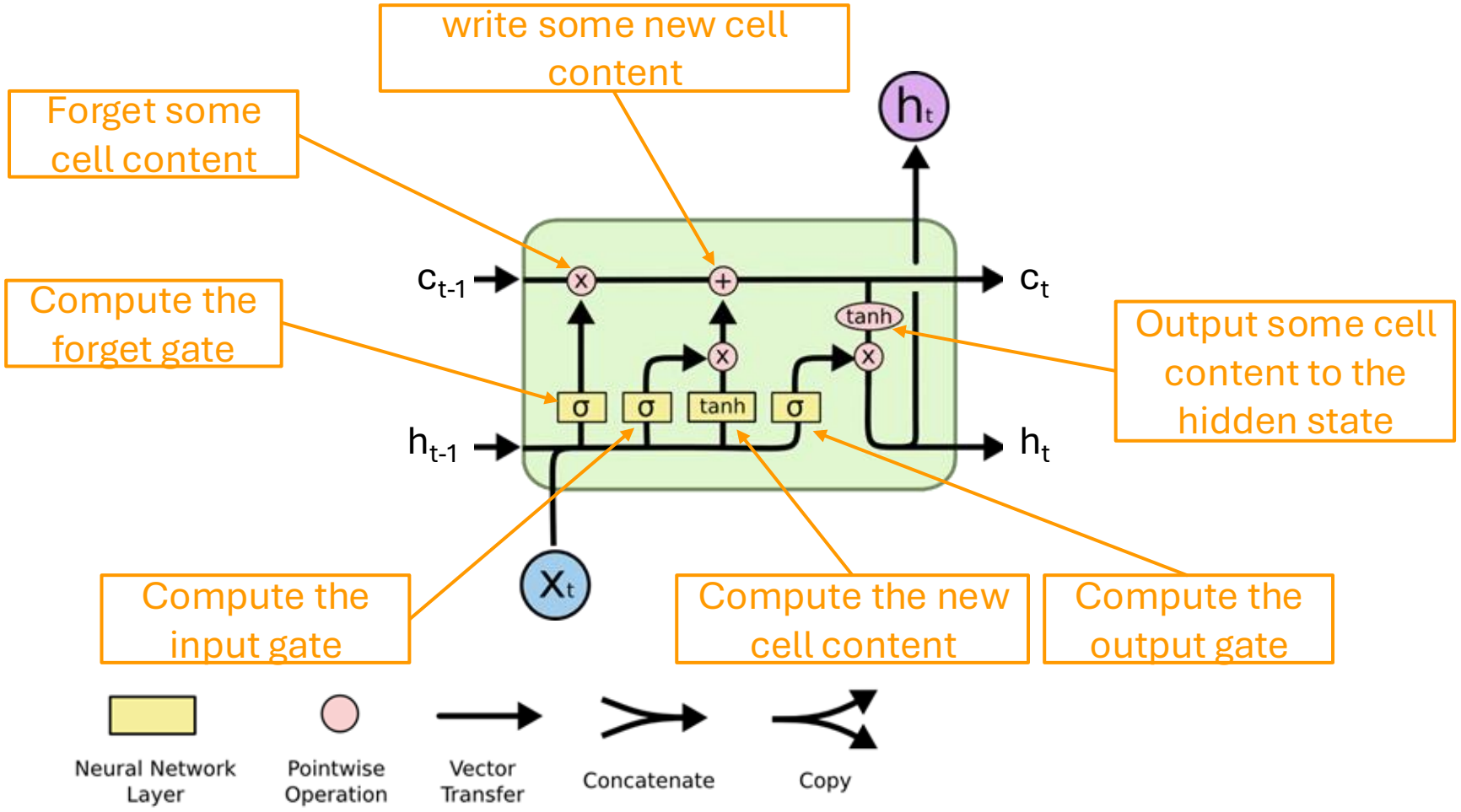
Think of the LSTM equations visually like this:



https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Long Short-Term Memory (LSTM)

Think of the LSTM equations visually like this:



https://colah.github.io/posts/2015-08-Understanding-LSTMs/

How does LSTM solve vanishing gradients?

- The LSTM architecture makes it easier for the RNN to preserve information over many timesteps
  - e.g. if the forget gate is set to remember everything on every timestep, then the info in the cell is preserved indefinitely
  - By contrast, it's harder for vanilla RNN to learn a recurrent weight matrix W_h that preserves info in hidden state

- LSTM doesn't *guarantee* that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies

Gated Recurrent Units (GRU)

Proposed by Cho et al. in 2014 as a simpler alternative to the LSTM
On each time step t, we have input $x^{(t)}$ and hidden state $h^{(t)}$ (no cell state)

Update gate: controls what parts of hidden state are updated vs preserved

$$\mathbf{u}^{(t)} = \sigma(W_u \mathbf{h}^{(t-1)} + U_u \mathbf{x}^{(t)} + \mathbf{b}_u)$$

Reset gate: controls what parts of previous hidden state are used to compute new content

$$\mathbf{r}^{(t)} = \sigma(W_r \mathbf{h}^{(t-1)} + U_r \mathbf{x}^{(t)} + \mathbf{b}_r)$$

New hidden state content: reset gate selects useful parts of prev. hidden state. Use this and current input to compute new hidden content

$$\tilde{\mathbf{h}}^{(t)} = \tanh(W_h(\mathbf{r}^{(t)} \circ \mathbf{h}^{(t-1)}) + U_h \mathbf{x}^{(t)} + \mathbf{b}_h)$$

Hidden state: update gate simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

$$\mathbf{h}^{(t)} = (1 - \mathbf{u}^{(t)}) \circ \mathbf{h}^{(t-1)} + \mathbf{u}^{(t)} \circ \tilde{\mathbf{h}}^{(t)}$$

How does this solve vanishing gradient?
Like LSTM, GRU makes it easier to retain info long-term (e.g. by setting update gate to 0)

"Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation", Cho et al. 2014, https://arxiv.org/pdf/1406.1078v3.pdf

LSTM vs GRU

- Researchers have proposed many gated RNN variants, but LSTM and GRU are the most widely-used.
- The biggest difference is that GRU is quicker to compute and has fewer parameters.
- There is no conclusive evidence that one consistently performs better than the other.
- LSTM is a good default choice (especially if your data has particularly long dependencies, or you have lots of training data).
- Rule of thumb: start with LSTM, but switch to GRU if you want something more efficient.

Is vanishing/exploding gradient just a RNN problem?

- No! It can be a problem for all neural architectures (including feed-forward and convolutional), especially deep ones.
  - Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small as it backpropagates
  - Thus lower layers are learnt very slowly (hard to train)
  - Solution: lots of new deep feedforward/convolutional architectures that add more direct connections (thus allowing the gradient to flow)

For example

- Residual connections aka "ResNet", also known as skip-connections
- The identity connection preserves information by default
- This makes deep network much easier to train

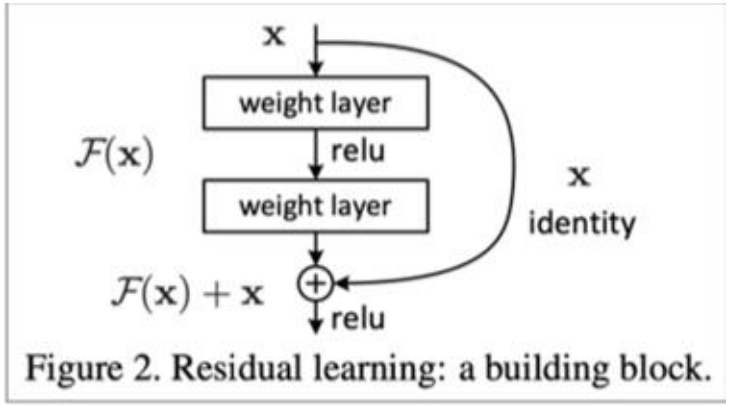


Figure 2. Residual learning: a building block.

"Deep Residual Learning for Image Recognition", He et al, 2015. https://arxiv.org/pdf/1512.03385.pdf
2023 Future Science Prize: http://www.futureprize.org/en/index.html

Is vanishing/exploding gradient just a RNN problem?

- No! It can be a problem for all neural architectures (including feed-forward and convolutional), especially deep ones.
  - Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small as it backpropagates
  - Thus lower layers are learnt very slowly (hard to train)
  - Solution: lots of new deep feedforward/convolutional architectures that add more direct connections (thus allowing the gradient to flow)

For example

- Dense connection aka “DenseNet”
- Connects each layer to every other layer in a feed-forward fashion
- Directly connect everything to everything

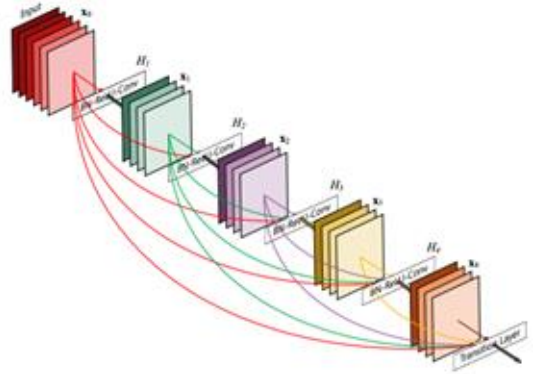


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

"Densely Connected Convolutional Networks", Huang et al, 2017. https://arxiv.org/pdf/1608.06993.pdf

Is vanishing/exploding gradient just a RNN problem?

- No! It can be a problem for all neural architectures (including feed-forward and convolutional), especially deep ones.
  - Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small as it backpropagates
  - Thus lower layers are learnt very slowly (hard to train)
  - Solution: lots of new deep feedforward/convolutional architectures that add more direct connections (thus allowing the gradient to flow)

For example

- Highway connections aka “HighwayNet”
- Similar to residual connections, but the identity connection vs the transformation layer is controlled by a dynamic gate
- Inspired by LSTMs, but applied to deep feedforward/convolutional networks

"Highway Networks", Srivastava et al, 2015. https://arxiv.org/pdf/1505.00387.pdf

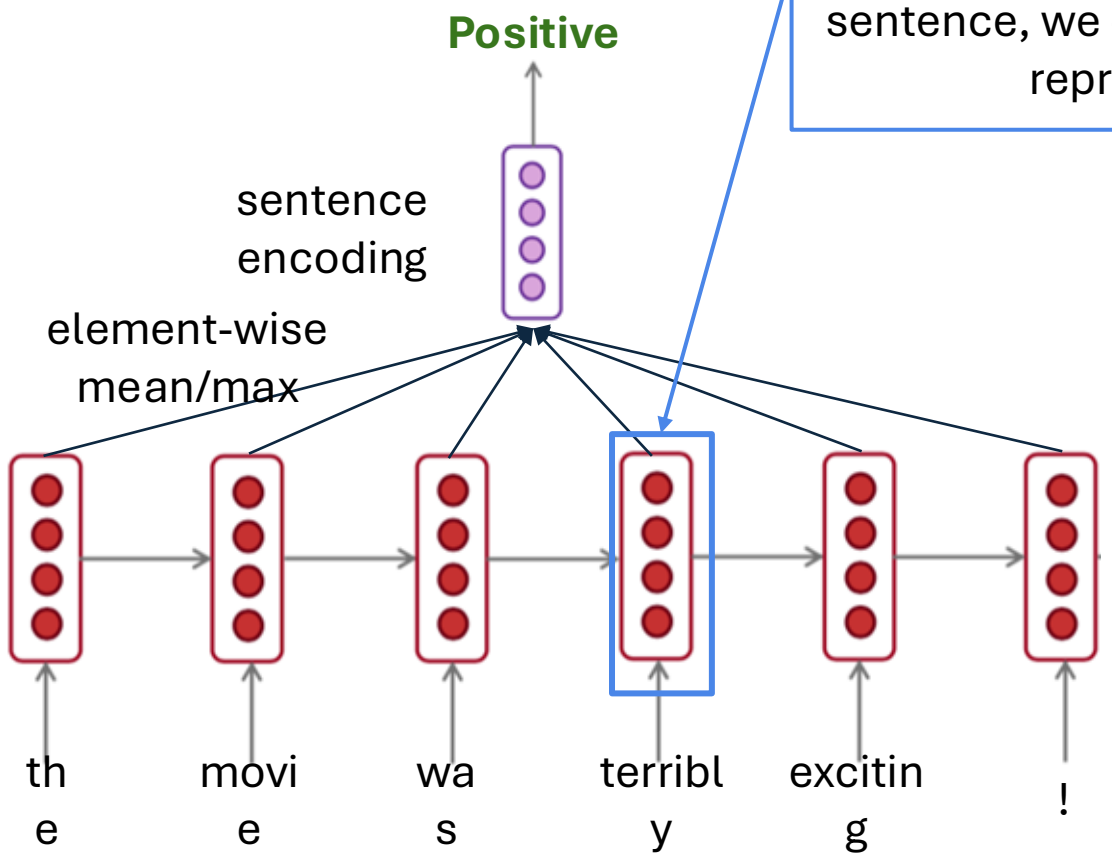
Is vanishing/exploding gradient just a RNN problem?

- No! It can be a problem for all neural architectures (including feed-forward and convolutional), especially deep ones.
  - Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small as it backpropagates
  - Thus lower layers are learnt very slowly (hard to train)
  - Solution: lots of new deep feedforward/convolutional architectures that add more direct connections (thus allowing the gradient to flow)

Conclusion:

- Though vanishing/exploding gradients are a general problem, RNNs are particularly unstable due to the repeated multiplication by the same weight matrix [Bengio et al, 1994]

Bidirectional RNNs: motivation
Task: sentiment classification



We can regard this hidden state as a representation of the word "terribly" in the context of this sentence, we call this a contextual representation

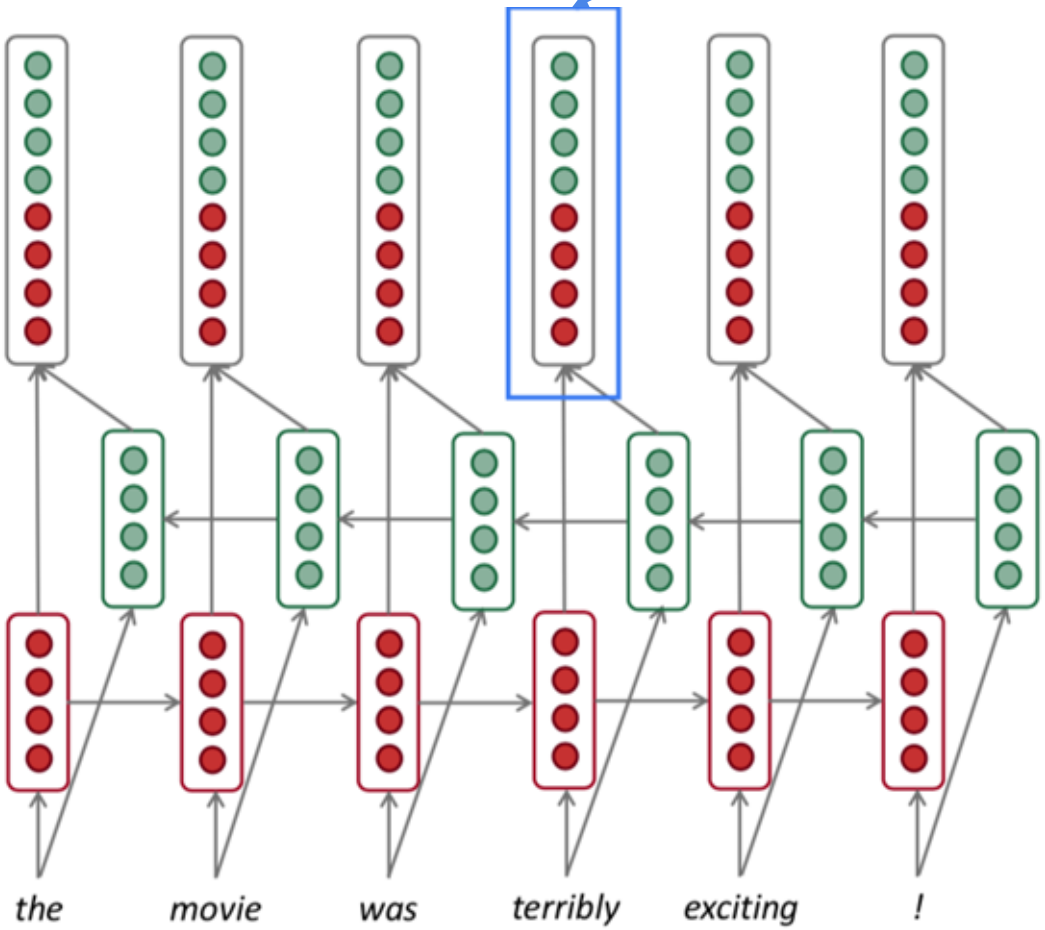
These contextual representations only contain information about the left context (e.g. "the movie was").

What about right context?

In this example, "exciting" is in the right context and this modifies the meaning of "terribly" (from negative to positive)

Bidirectional RNNs

This contextual representation of "terribly" has both left and right context!



Concatenated hidden states

Backward RNN

Forward RNN

Bidirectional RNNs

This is a general notation to mean "compute one forward step of the RNN" – it could be a vanilla, LSTM or GRU computation.

Forward RNN

$$\overrightarrow{\mathbf{h}}^{(t)} = \mathrm{RNN}_{\mathrm{FW}}(\overrightarrow{\mathbf{h}}^{(t-1)}, \mathbf{x}^{(t)})$$

Backward RNN

$$\overleftarrow{\mathbf{h}}^{(t)} = \mathrm{RNN}_{\mathrm{BW}}(\overleftarrow{\mathbf{h}}^{(t+1)}, \mathbf{x}^{(t)})$$

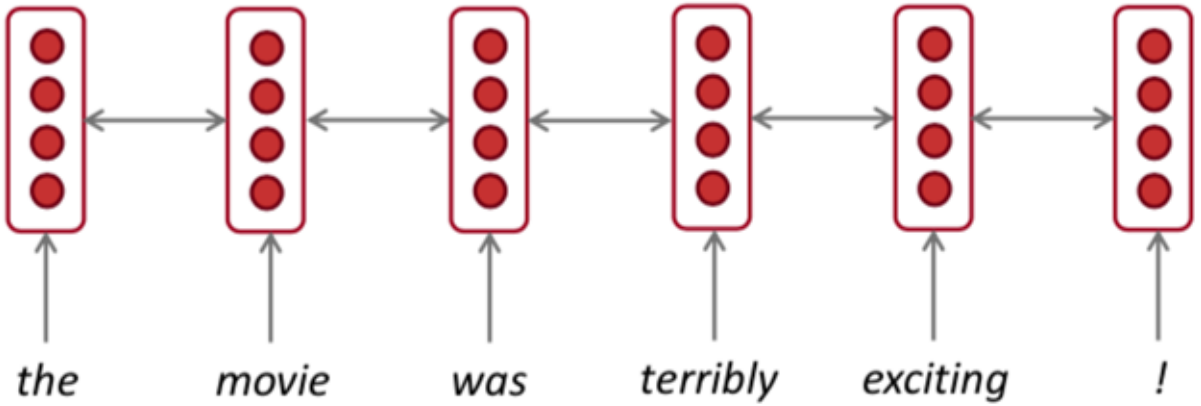
Generally, these two RNNs have separate weights

Concatenated hidden states

$$\mathbf{h}^{(t)} = [\overrightarrow{\mathbf{h}}^{(t)}; \overleftarrow{\mathbf{h}}^{(t)}]$$

We regard this as "the hidden state" of a bidirectional RNN. This is what we pass on to the next parts of the network.

Bidirectional RNNs: simplified diagram



- The two-way arrows indicate bidirectionality
- The depicted hidden states are assumed to be the concatenated forwards+backwards states.

Bidirectional RNNs

- Note: bidirectional RNNs are only applicable if you have access to the entire input sequence.
  - They are not applicable to Language Modeling, because in LM you only have left context available.

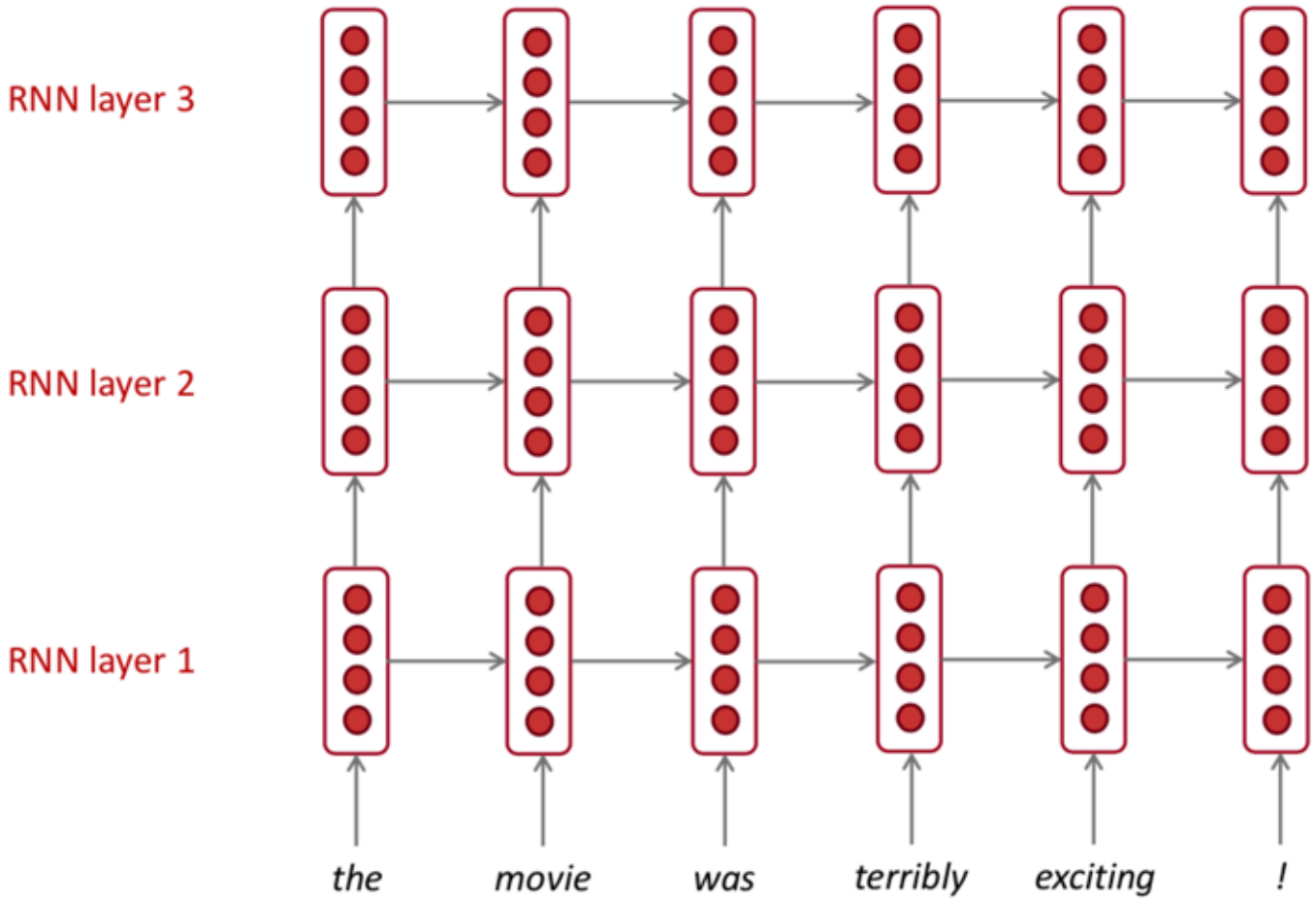
- If you do have entire input sequence (e.g. any kind of encoding), bidirectionality is powerful (you should use it by default).

- For example, BERT (**Bidirectional** Encoder Representations from Transformers) is a powerful pretrained contextual representation system built on bidirectionality.

Multi-layer RNNs

- RNNs are already “deep” on one dimension (they unroll over many timesteps)
- We can also make them “deep” in another dimension by applying multiple RNNs – this is a multi-layer RNN.
- This allows the network to compute more complex representations
- The lower RNNs should compute lower-level features and the higher RNNs should compute higher-level features.
- Multi-layer RNNs are also called *stacked RNNs*.

Multi-layer RNNs



The hidden states from RNN layer i are the inputs to RNN layer i+1

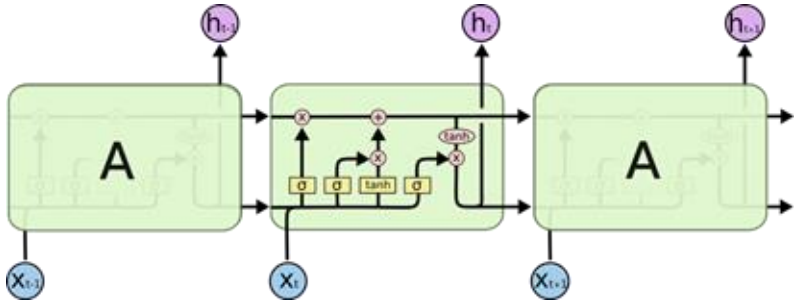
Multi-layer RNNs in practice

- High-performing RNNs are often multi-layer (but aren't as deep as convolutional or feed-forward networks)

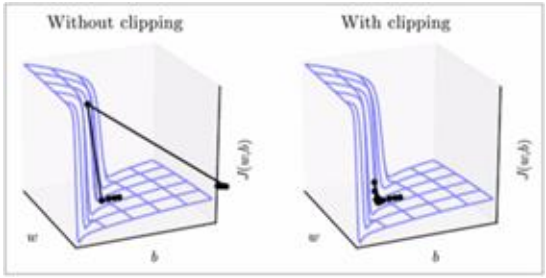
- For example: In a 2017 paper, Britz et al find that for Neural Machine Translation, 2 to 4 layers is best for the encoder RNN, and 4 layers is best for the decoder RNN
  - However, skip-connections/dense-connections are needed to train deeper RNNs (e.g. 8 layers)
- Transformer-based networks (e.g. BERT) can be up to 24 layers

Summary of RNN

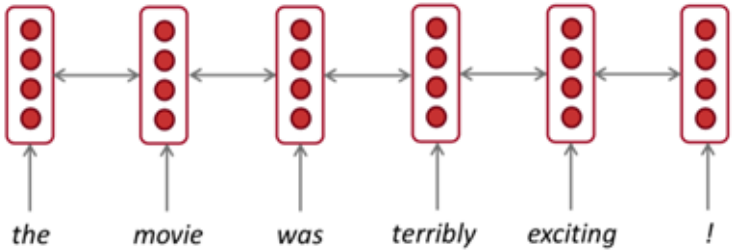
- What are the practical takeaways?



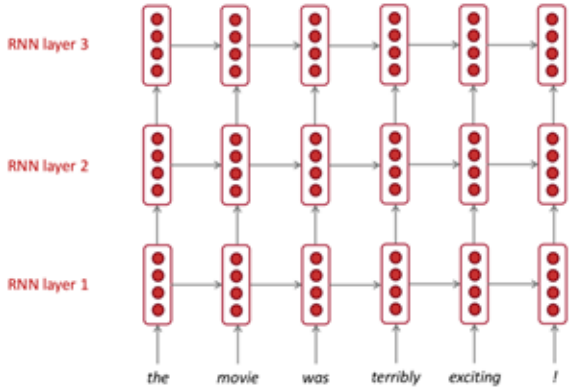
1. LSTM are powerful but GRUs are faster



2. Clip your gradients



3. Use bidirectionality when possible



4. Multi-layer RNNs are powerful, but you might need skip/dense-connections if it's deep

Readings

- **CH10 DL**; **CH15-16 NNLP**
- Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. 1997 (LSTM)
- Cho et al. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. 2014

Development of LM

Stages	Details
Statistical LM	• Developed based on **statistical** learning methods in 1990s. • N-gram LMs, such as bigram and trigram. • Limitation: difficult to accurately estimate high-order language models; specially designed smoothing strategies are used to alleviate the data sparsity problem
Neural LM	• Characterize the probability of word sequences by **neural networks**, e.g. RNN • Introduced the concept of **distributed representation of words**, word2vec • Initiated the use of language models for representation learning
Pre-trained LM (PLM)	• **Pre-training first and then fine-tuning** on specific downstream task, allows to capture context-aware word representation • BERT: pretraining with specially designed tasks on large-scale unlabeled data, which is very effective to serve a general-purpose semantic features and improves the performance of NLP tasks. • Inspired many follow-up work: with different architectures or improved pre-training strategies.
Large LM (LLM)	• **Scaling PLM in model size** often leads to an **emergent** model capacity. • For example, GPT-3 can solve few-shot tasks through in-context learning, while GPT-2 cannot do well. • The term LLM is coined for the large-sized PLMs. • An application of LLMs is ChatGPT that adapts the LLMs from the GPT series for dialogue. • Followed with a sharp release of LLMs: Bard, Claude 2, LLaMA, Falcon, etc.

Part 3

Convolutional Neural Networks (CNN) for Text

Convolutional Neural Networks

- Originally, convolutional neural networks are specialized networks for application in **computer vision**
  - Accept images as raw input (preserving spatial information)
  - Build up (learn) a hierarchy of features (no hand-crafted features necessary).

Why CNN for Image

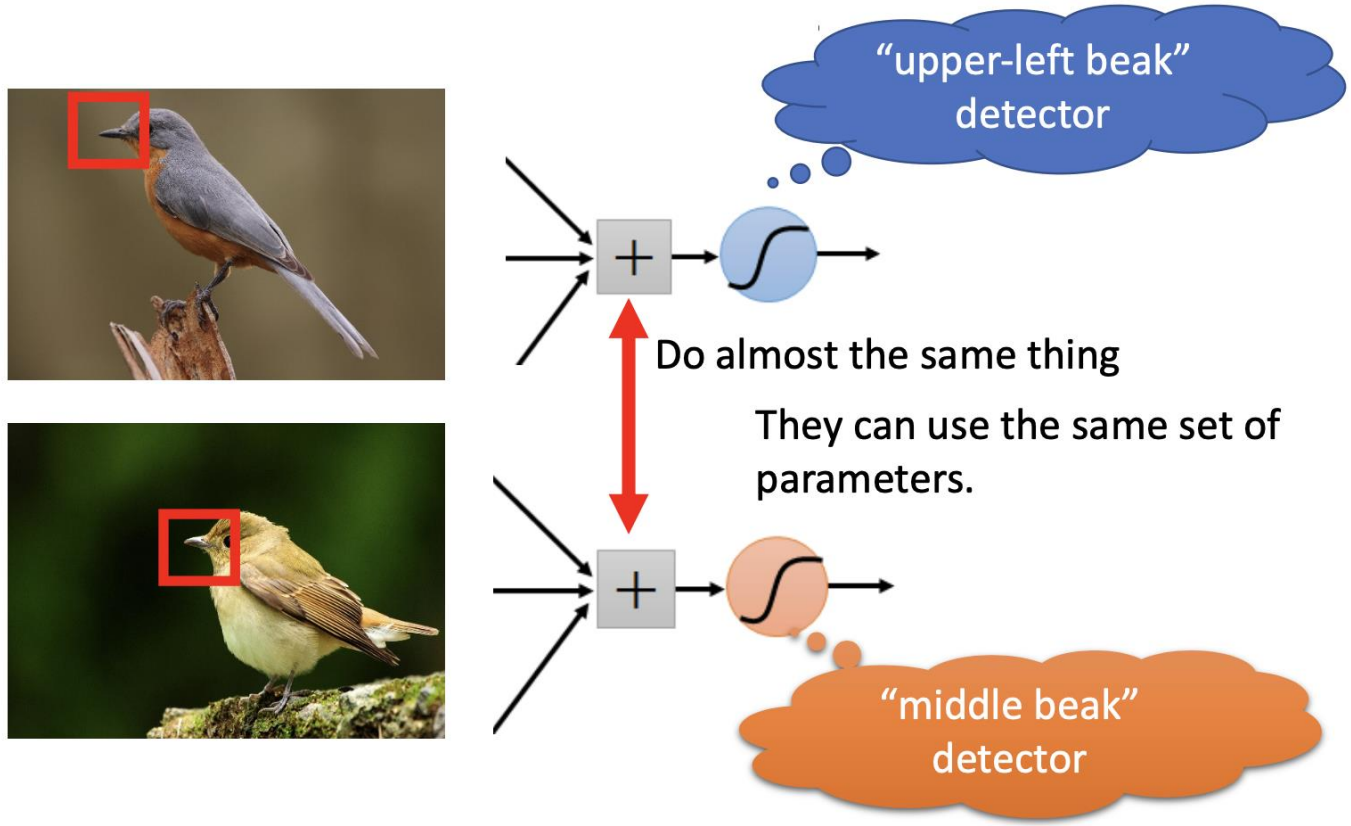
- Some patterns are much smaller than the whole image.

A neuron does not have to see the whole image to discover the pattern.

Connecting to small region with less parameters

Why CNN for Image

- The same patterns appear in different regions.



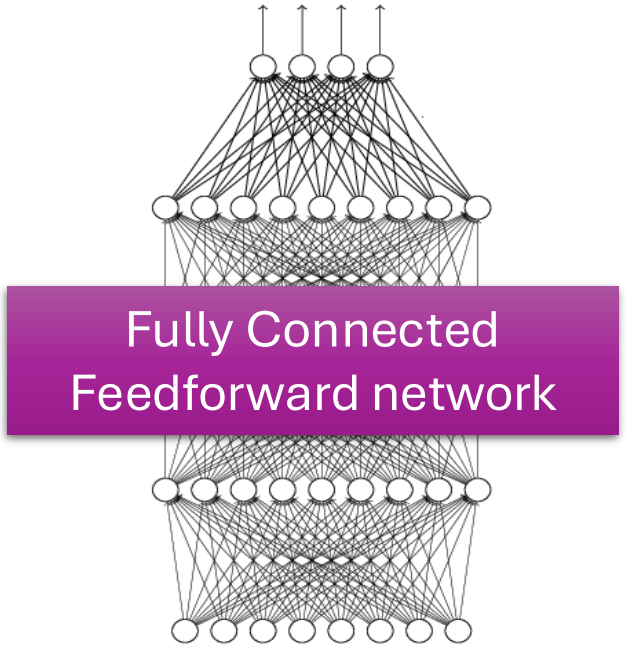
Why CNN for Image

- Subsampling the pixels will not change the object

We can subsample the pixels to make image smaller

→ Less parameters for the network to process the image

The Whole CNN



The Whole CNN

Property 1

- Some patterns are much smaller than the whole image

Property 2

- The same patterns appear in different regions.

Property 3

- Subsampling the pixels will not change the object

Convolution

Max Pooling

Convolution

Max Pooling

Can repeat many times

Flatten

CNN – Convolution

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Property 1: Some patterns are much smaller than the whole image

Those are the network parameters to be learned.

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Matrix

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

Matrix

Each filter detects a small pattern (3 x 3).

CNN – Convolution

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

3

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

3 -1

CNN – Convolution

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

3

6 x 6 image

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

3 -3

6 x 6 image

We set stride=1 below

CNN – Convolution

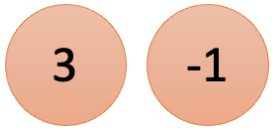
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



CNN – Convolution

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

CNN – Convolution

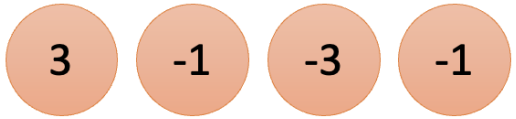
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



CNN – Convolution

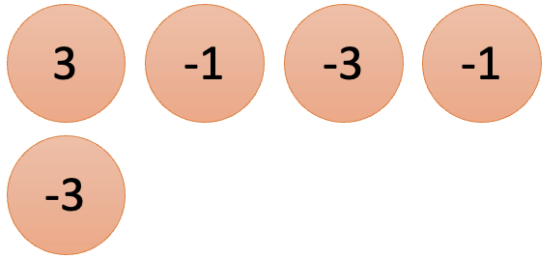
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



CNN – Convolution

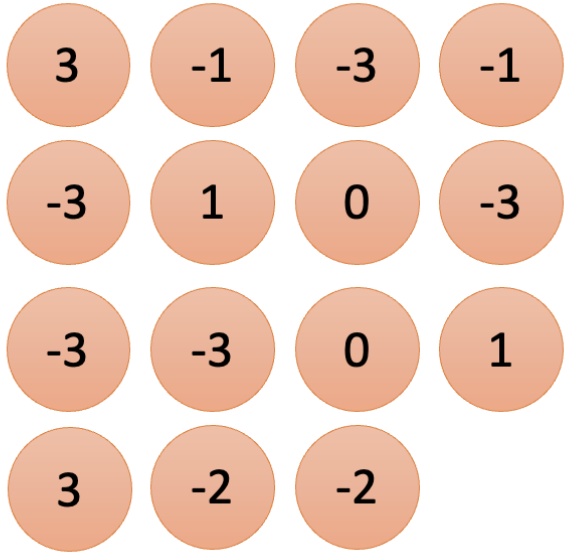
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



CNN – Convolution

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

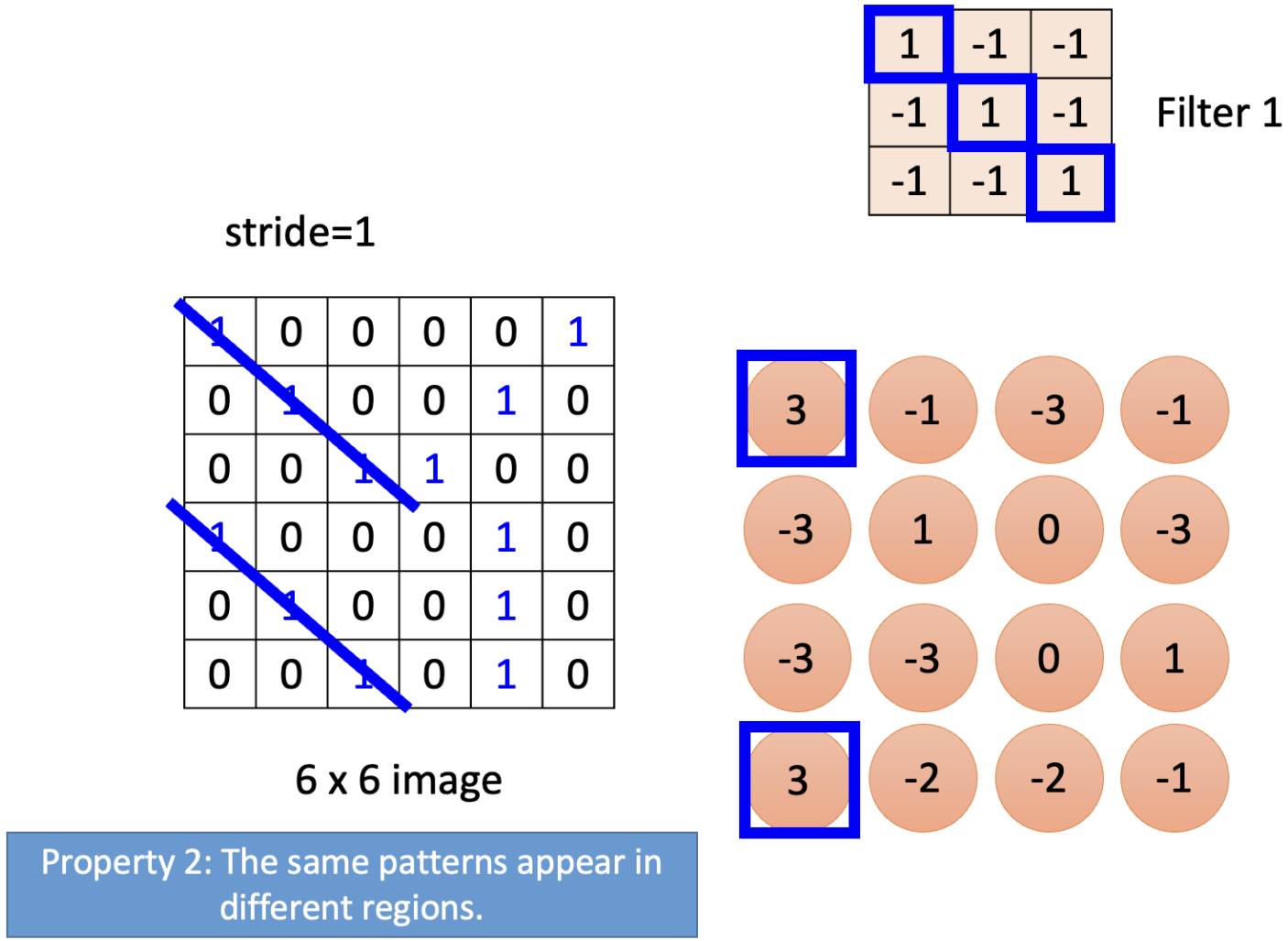
stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

CNN – Convolution



CNN – Convolution

-1	1	-1
-1	1	-1
-1	1	-1

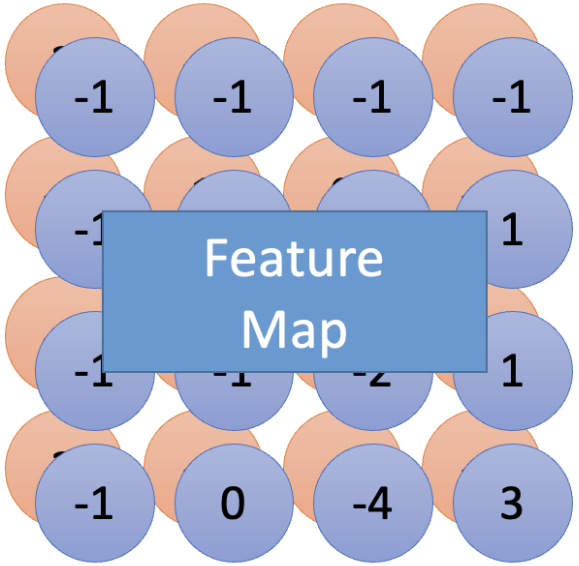
Filter 2

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

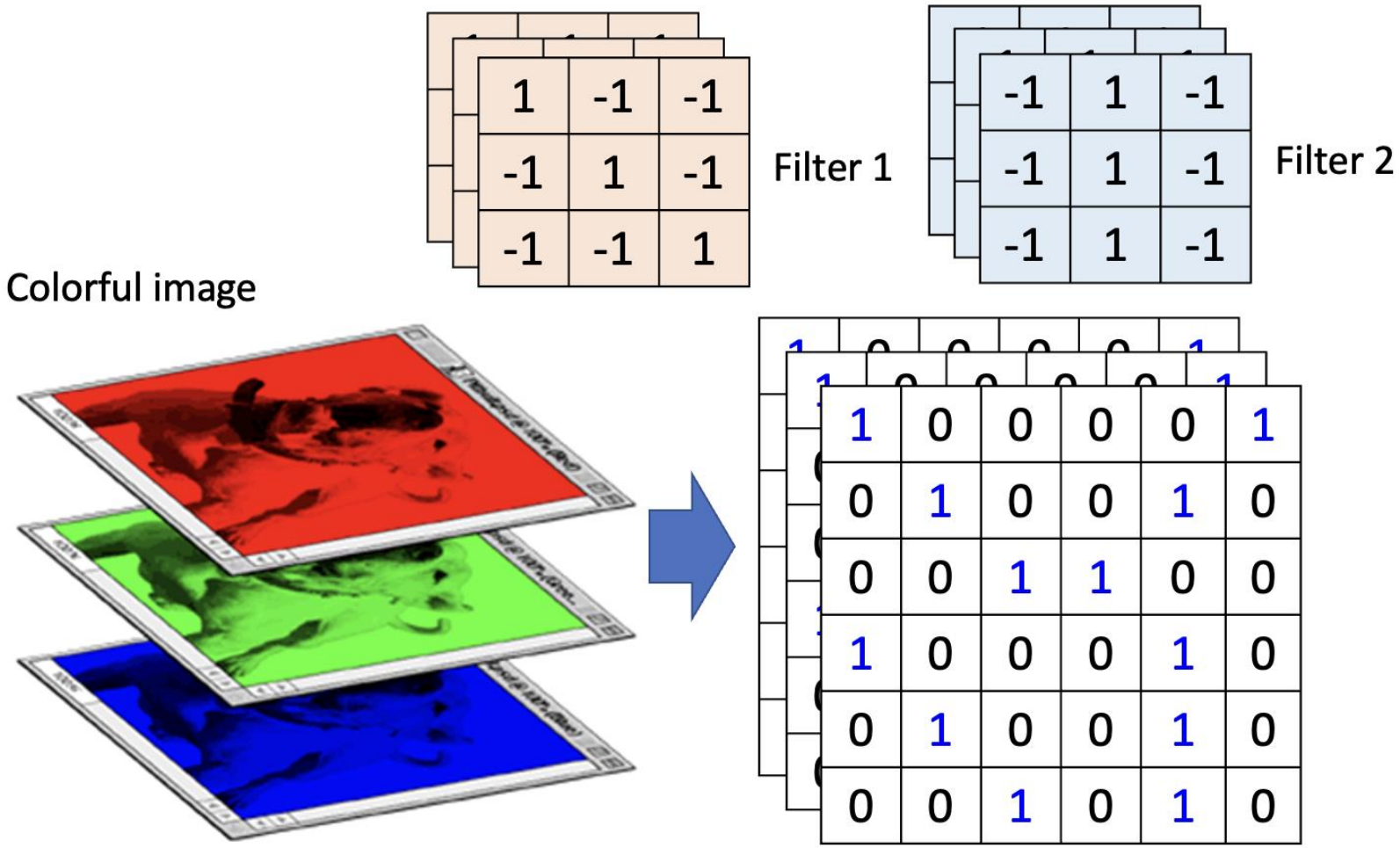
6 x 6 image

Do the same process for every filter

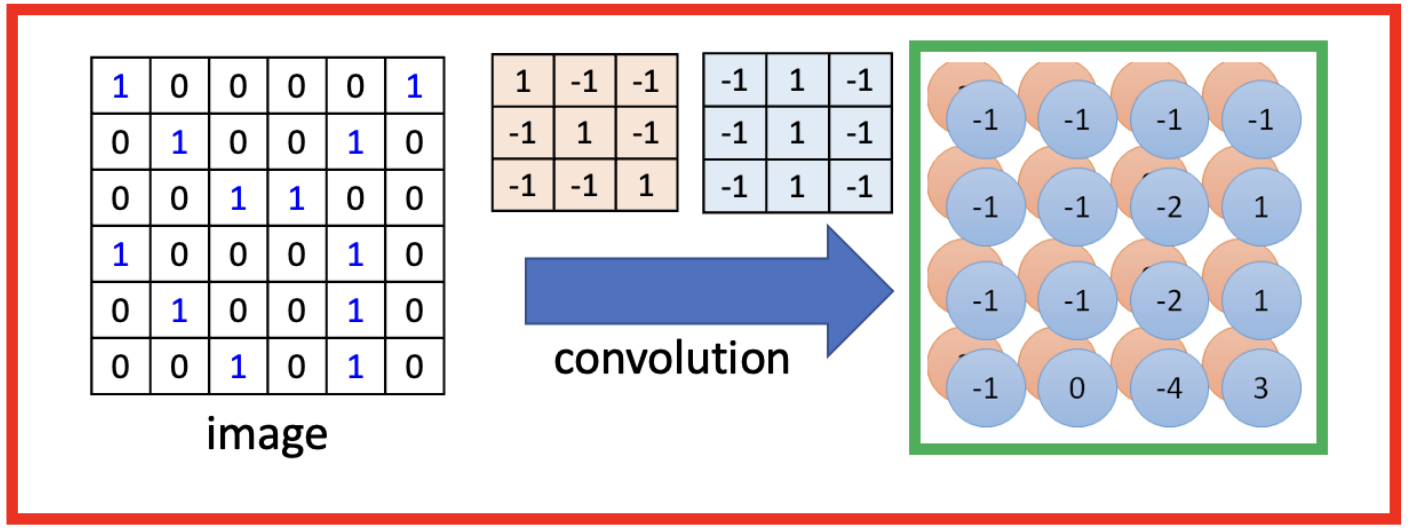


4 x 4 image

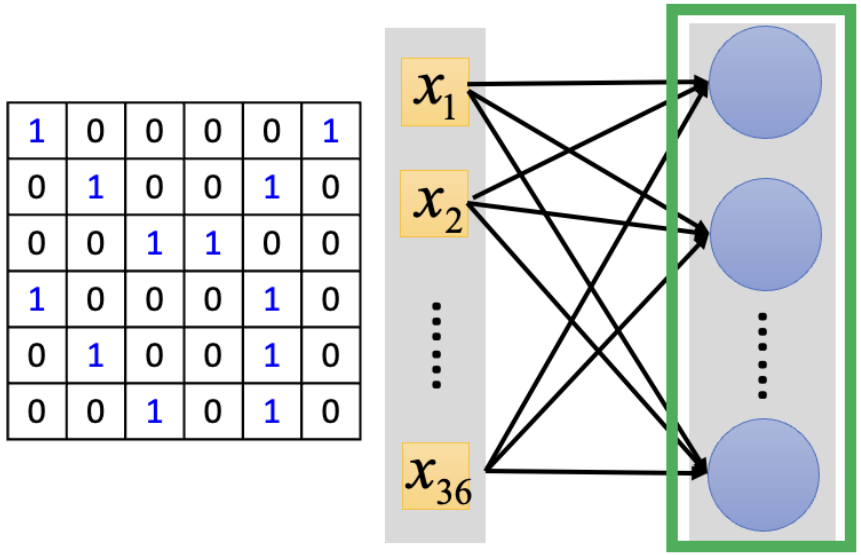
CNN – Colorful Image

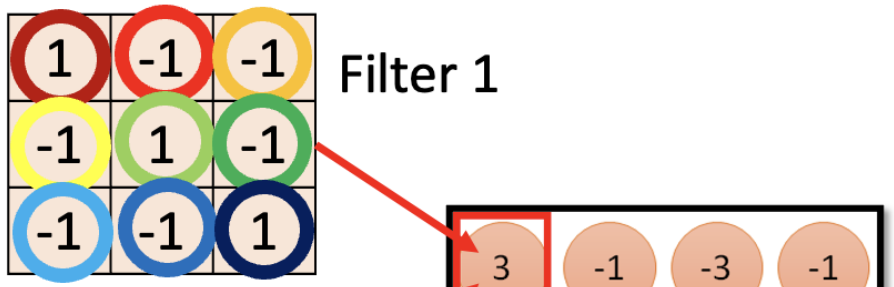


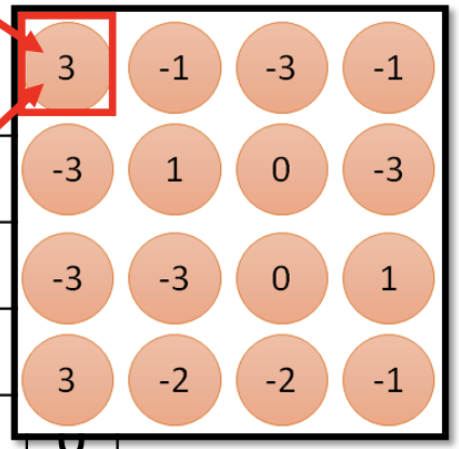
Convolution v.s. Fully Connected



Fully-connected



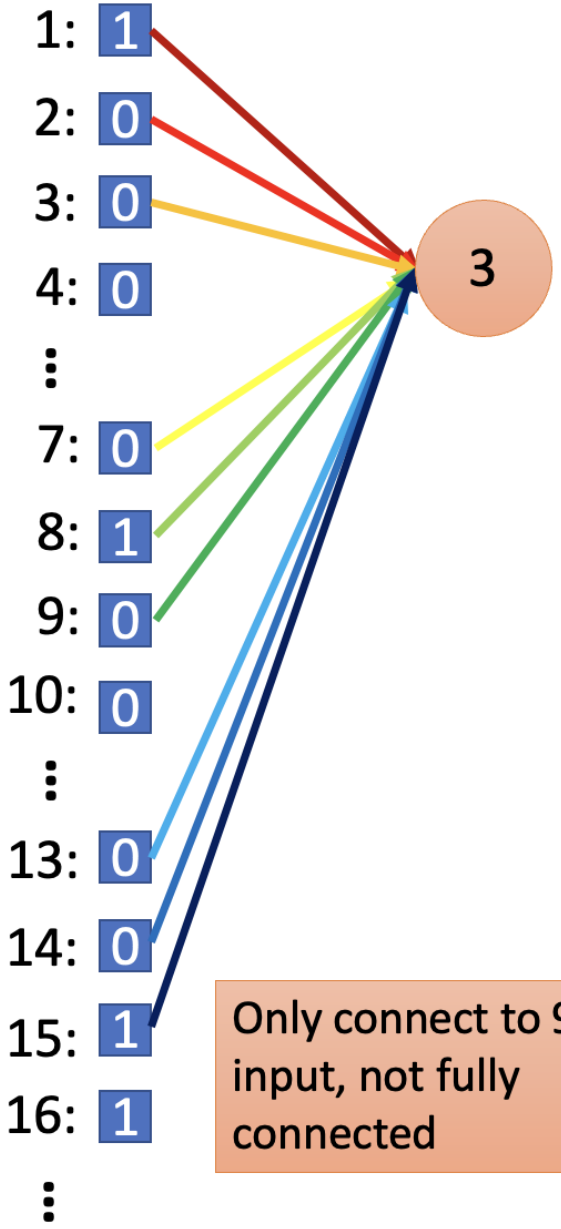




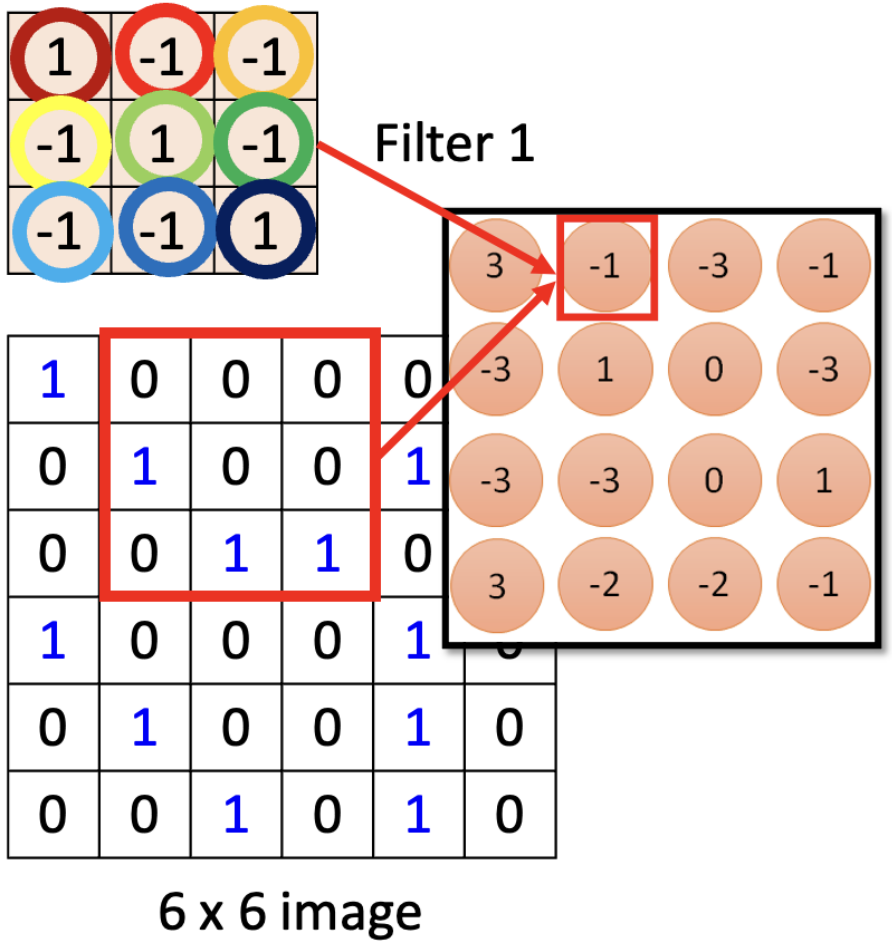
1	0	0	0	0	
0	1	0	0	1	
0	0	1	1	0	
1	0	0	0	1	
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Less parameters!

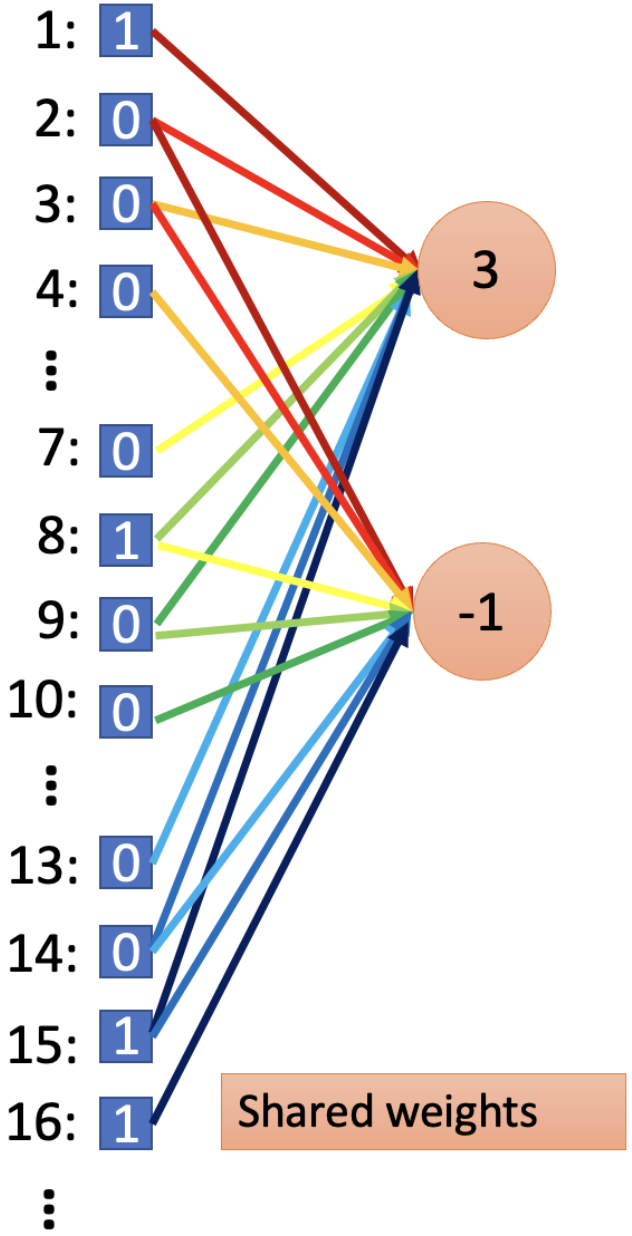


Only connect to 9 input, not fully connected



Less parameters!

Even less parameters!



The Whole CNN

Convolution

Max Pooling

Convolution

Max Pooling

Can repeat many times

Flatten

Fully Connected Feedforward network

cat dog .....

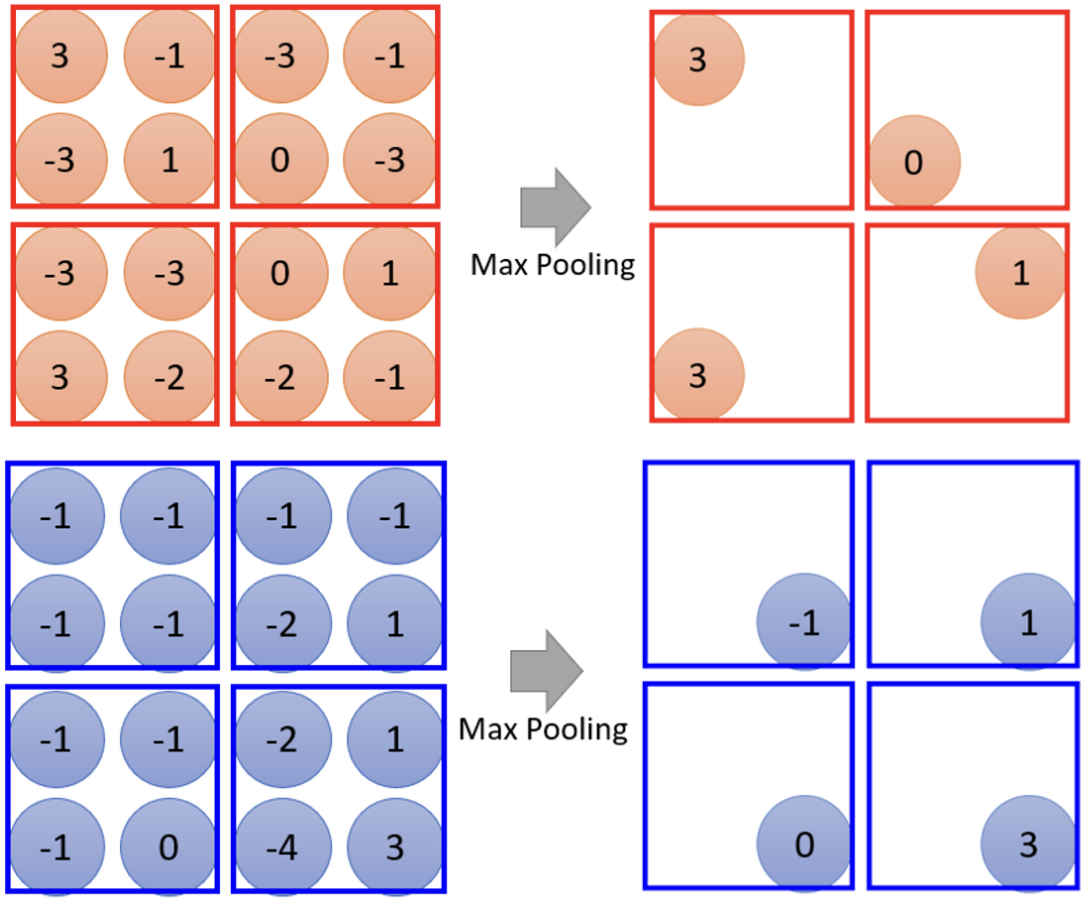
CNN – Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

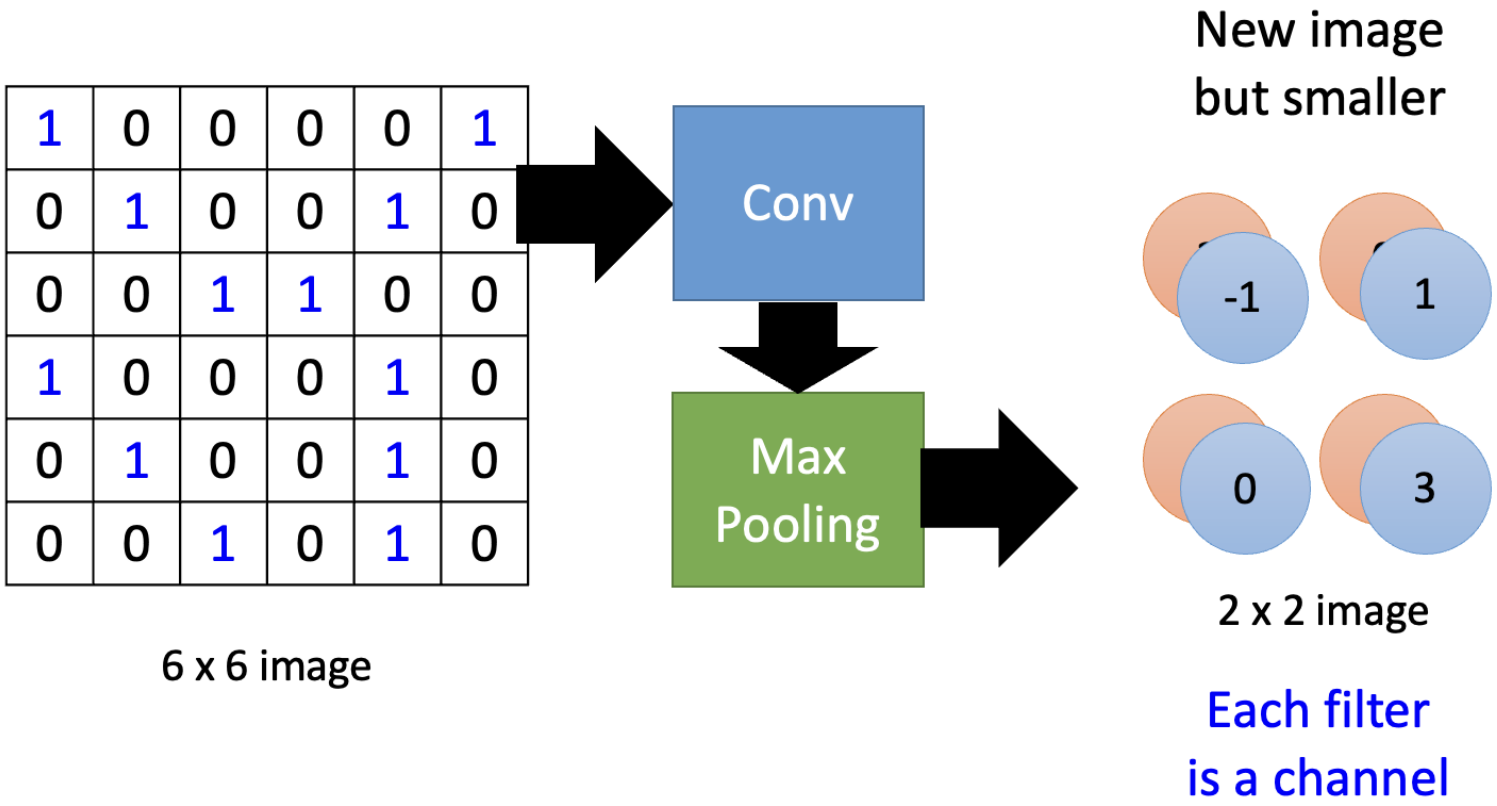
Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

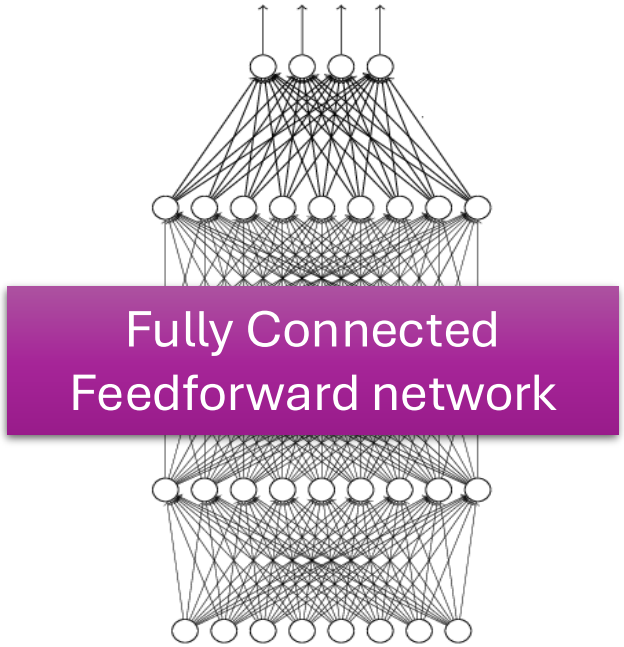
Filter 2



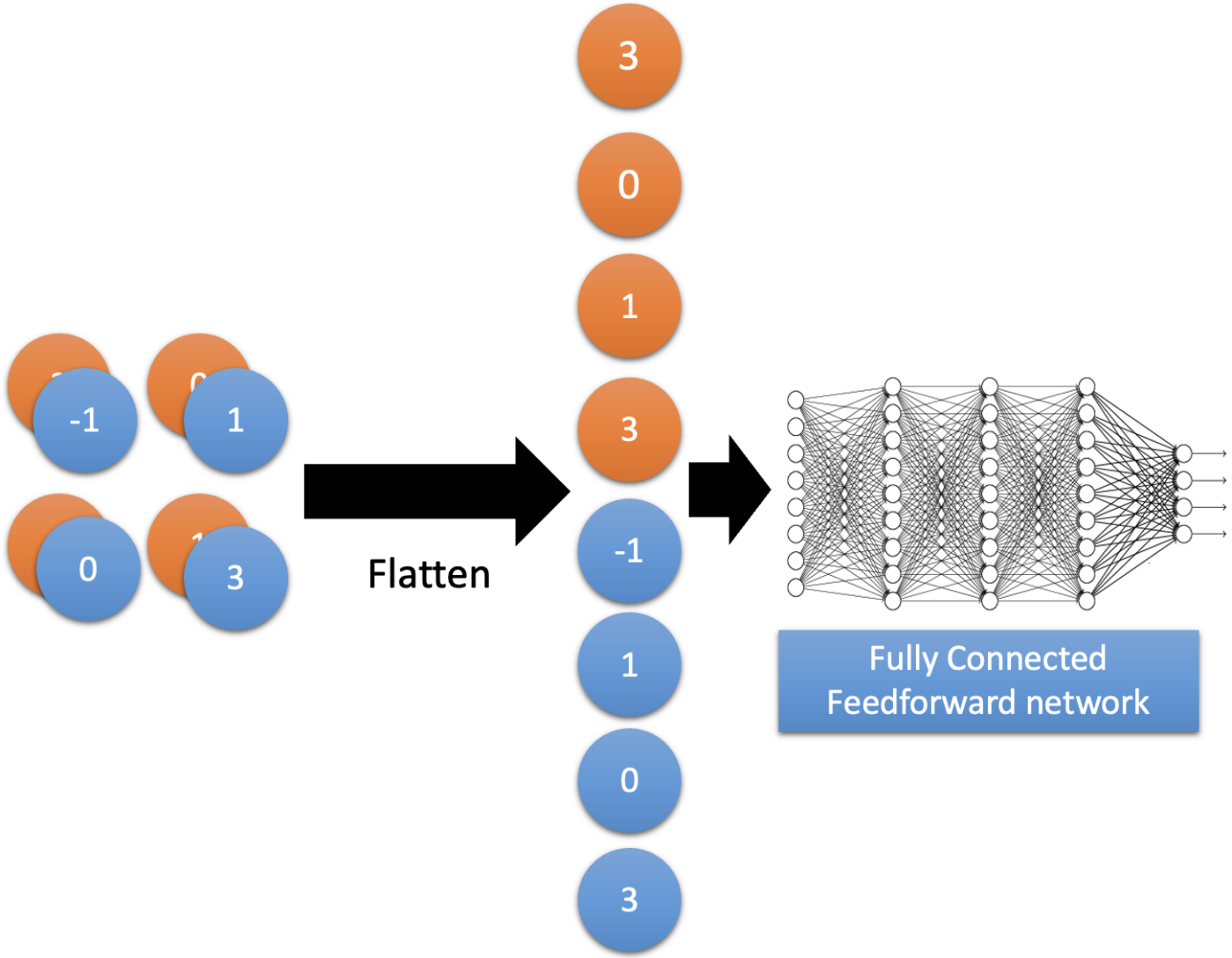
CNN – Max Pooling



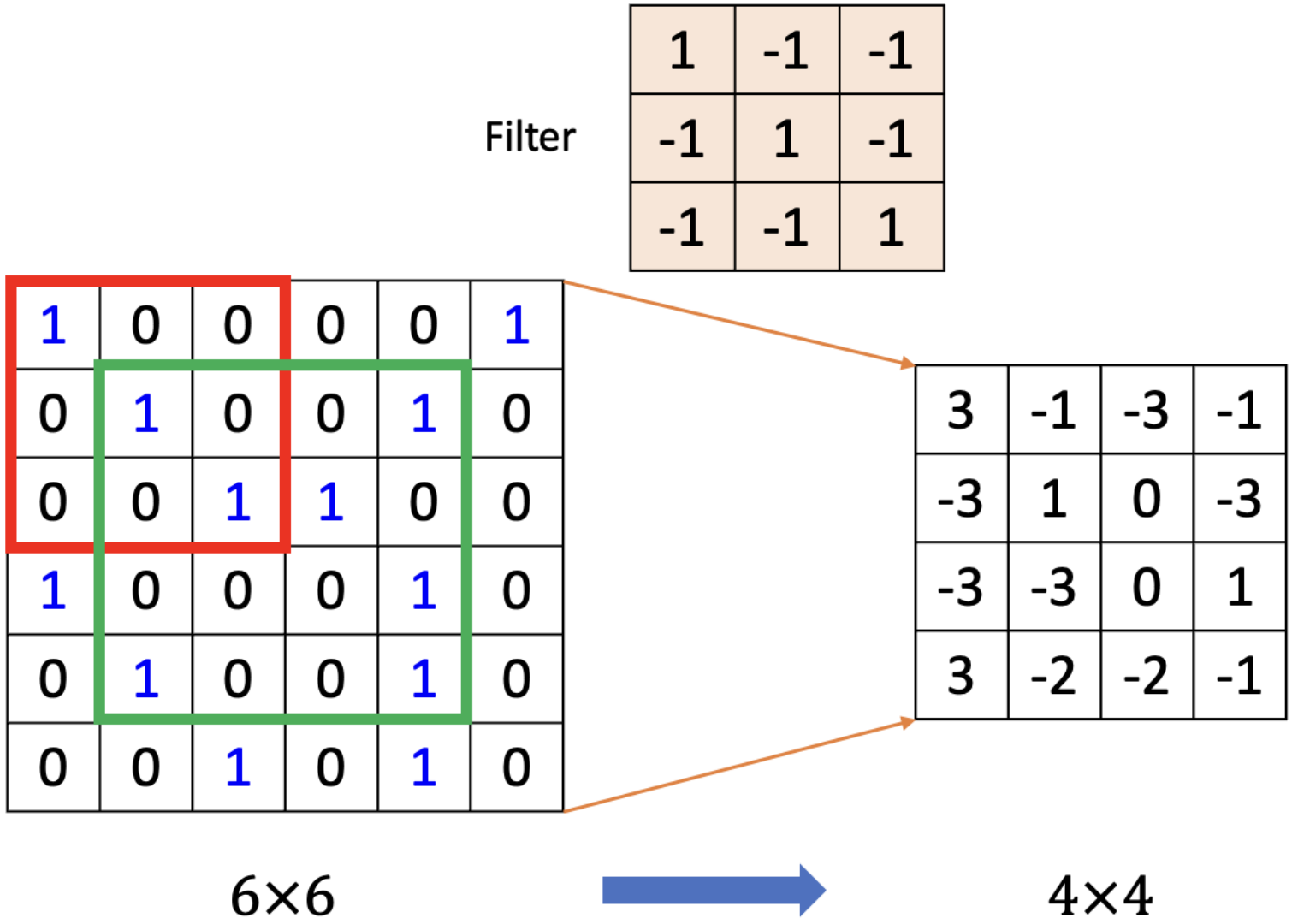
The Whole CNN



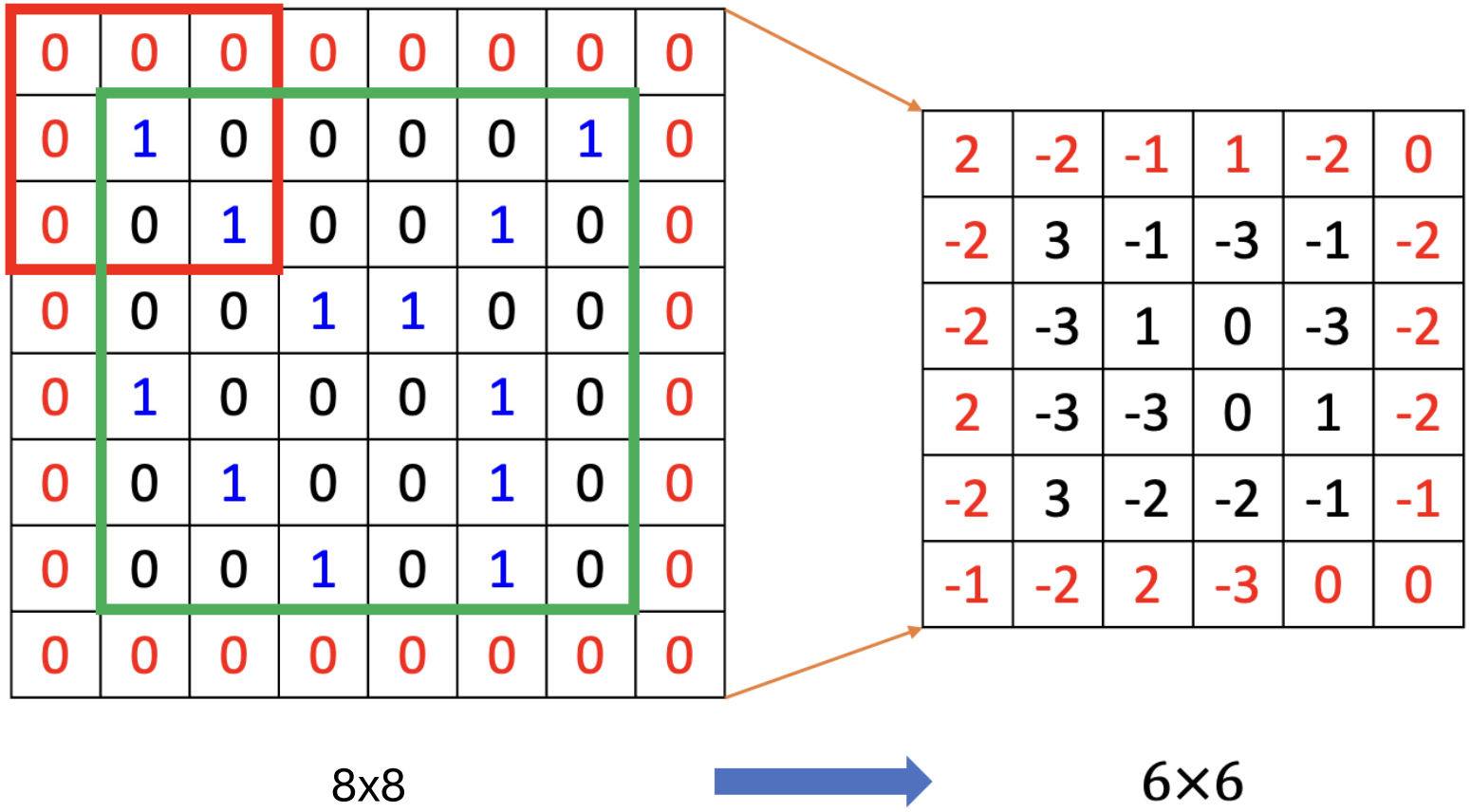
Flatten


-1
1
0
3
Flatten
3
0
1
3
-1
1
0
3
Fully Connected
Feedforward network

Without padding

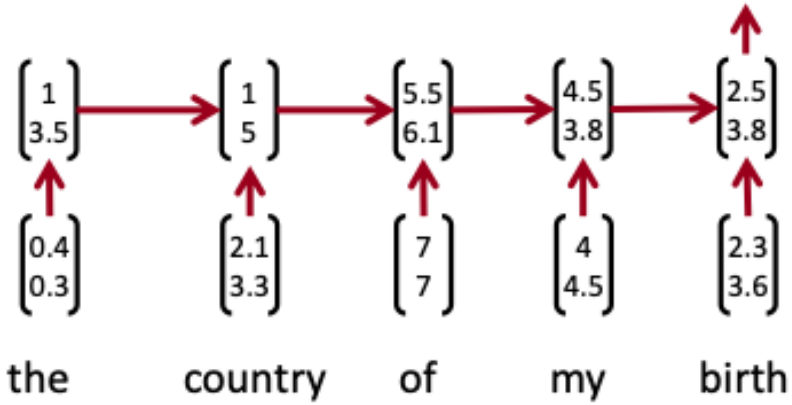


With Padding



Why might CNN work for text?

- Recurrent neural nets cannot capture phrases without prefix context
- Often capture too much of last words in final vector



- E.g. softmax is often only calculated at the last step

From RNNs to CNNs

- Main CNN idea:
  - What if we compute vectors for every possible word subsequences of a certain length?

- Example: “tentative deal reached to keep government open” computes vectors for:
  - Tentative deal reached, deal reached to, reached to keep, to keep government, keep government open

- Regardless of whether phrase is grammatical
- Not very linguistically or cognitively plausible
- Then group them afterwards

A 1D convolution for text

tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3

t,d,r	−1.0
d,r,t	−0.5
r,t,k	−3.6
t,k,g	−0.2
k,g,o	0.3

Apply a filter (kernel) of

3	1	2	−3
−1	2	1	−3
1	1	−1	1

An animation example:
https://cezannec.github.io/CNN_Text_Classification/

A 1D convolution for text with padding

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Ø,t,d	−0.6
t,d,r	−1.0
d,r,t	−0.5
r,t,k	−3.6
t,k,g	−0.2
k,g,o	0.3
g,o,Ø	−0.5

Apply a filter (kernel) of

3	1	2	−3
−1	2	1	−3
1	1	−1	1

3 channel 1D convolution with padding

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Ø,t,d	−0.6	0.2	1.4
t,d,r	−1.0	1.6	−1.0
d,r,t	−0.5	−0.1	0.8
r,t,k	−3.6	0.3	0.3
t,k,g	−0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,Ø	−0.5	−0.9	0.1

Apply 3 filters of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

conv1d, padded with max pooling over time

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Ø,t,d	−0.6	0.2	1.4
t,d,r	−1.0	1.6	−1.0
d,r,t	−0.5	−0.1	0.8
r,t,k	−3.6	0.3	0.3
t,k,g	−0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,Ø	−0.5	−0.9	0.1
max p	0.3	1.6	1.4

Apply 3 filters of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

conv1d, padded with ave pooling over time

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Ø,t,d	−0.6	0.2	1.4
t,d,r	−1.0	1.6	−1.0
d,r,t	−0.5	−0.1	0.8
r,t,k	−3.6	0.3	0.3
t,k,g	−0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,Ø	−0.5	−0.9	0.1
ave p	−0.87	0.26	0.53

Apply 3 filters of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

Convolution with stride = 2

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Ø,t,d	−0.6	0.2	1.4
d,r,t	−0.5	−0.1	0.8
t,k,g	−0.2	0.1	1.2
g,o,Ø	−0.5	−0.9	0.1

Apply 3 filters of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

Single layer CNN for sentence classification

- Yoon Kim (2014): Convolutional Neural Networks for Sentence Classification. EMNLP 2014.
  - Paper: https://arxiv.org/pdf/1408.5882.pdf
  - Code: https://github.com/yoonkim/CNN_sentence [Theano!]

- Goal: Sentence classification:
  - Mainly positive or negative sentiment of a sentence
  - Other tasks like:
    - Subjective or objective language sentence
    - Question classification: about person, location, number,...

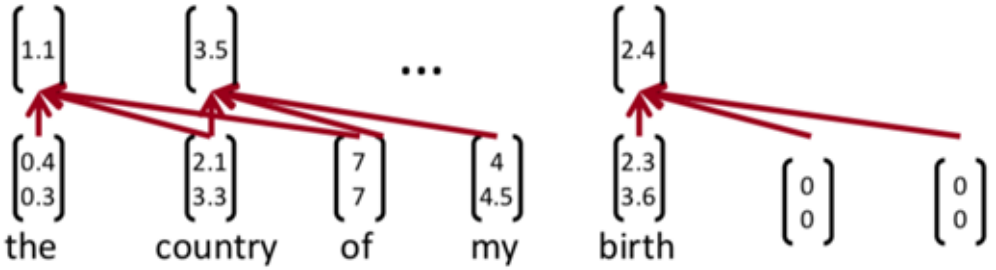
Single layer CNN

- Filter w is applied to all possible windows (concatenated vectors)
- To compute feature (one channel) for CNN layer:

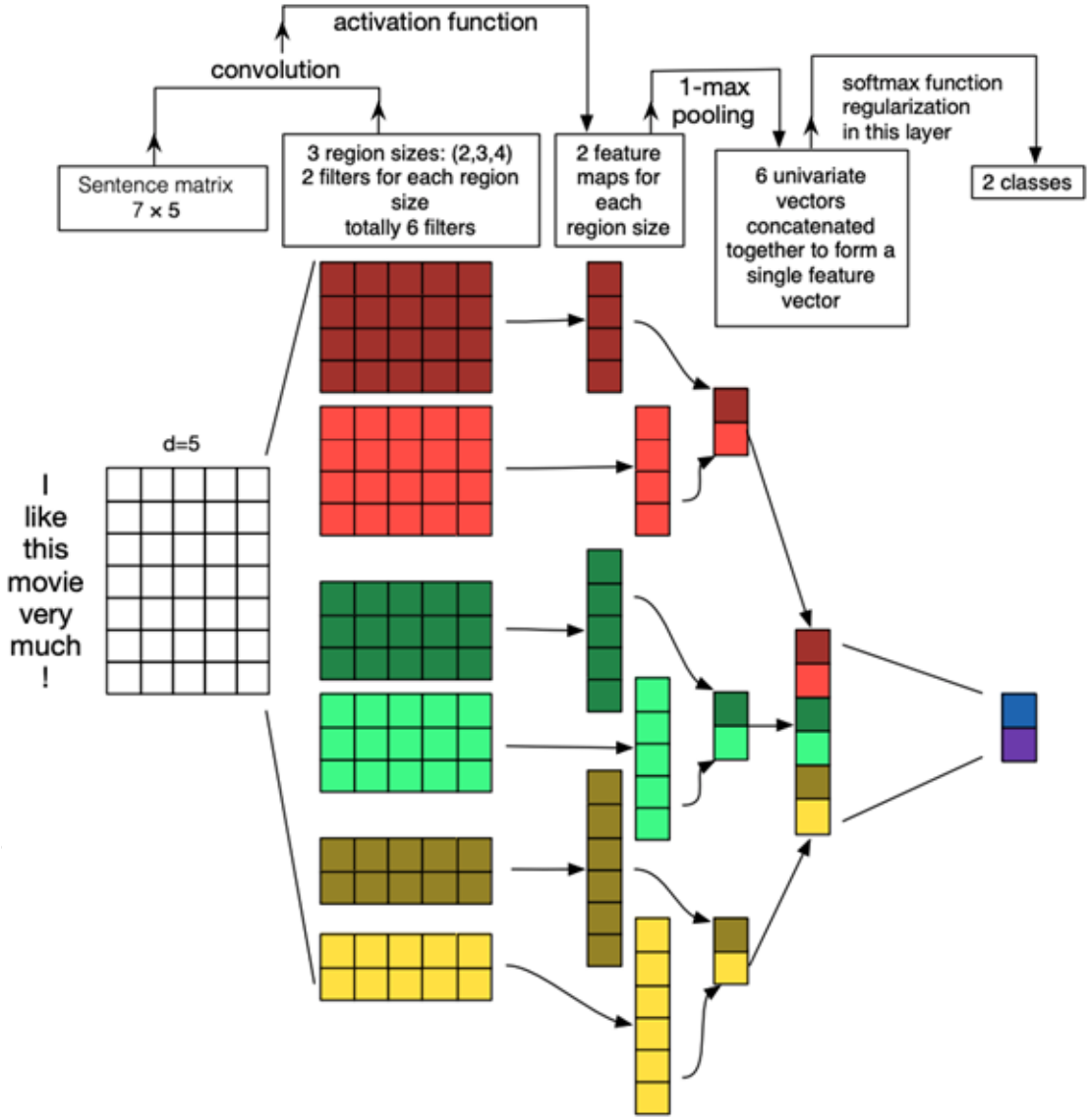
$$c_i = f(\mathbf{w}^\top \mathbf{x}_{i:i+h-1} + b)$$

- Sentence $x_{1:n} = x_1 \oplus x_2 \oplus \ldots \oplus x_n$
- All possible windows of length h: $\{x_{1:h}, x_{2:h+1}, \ldots, x_{n-h+1:n}\}$

- Result is a feature map: $c=[c_1, c_2, \ldots, c_{n-h+1}] \in \mathbb{R}^{n-h+1}$

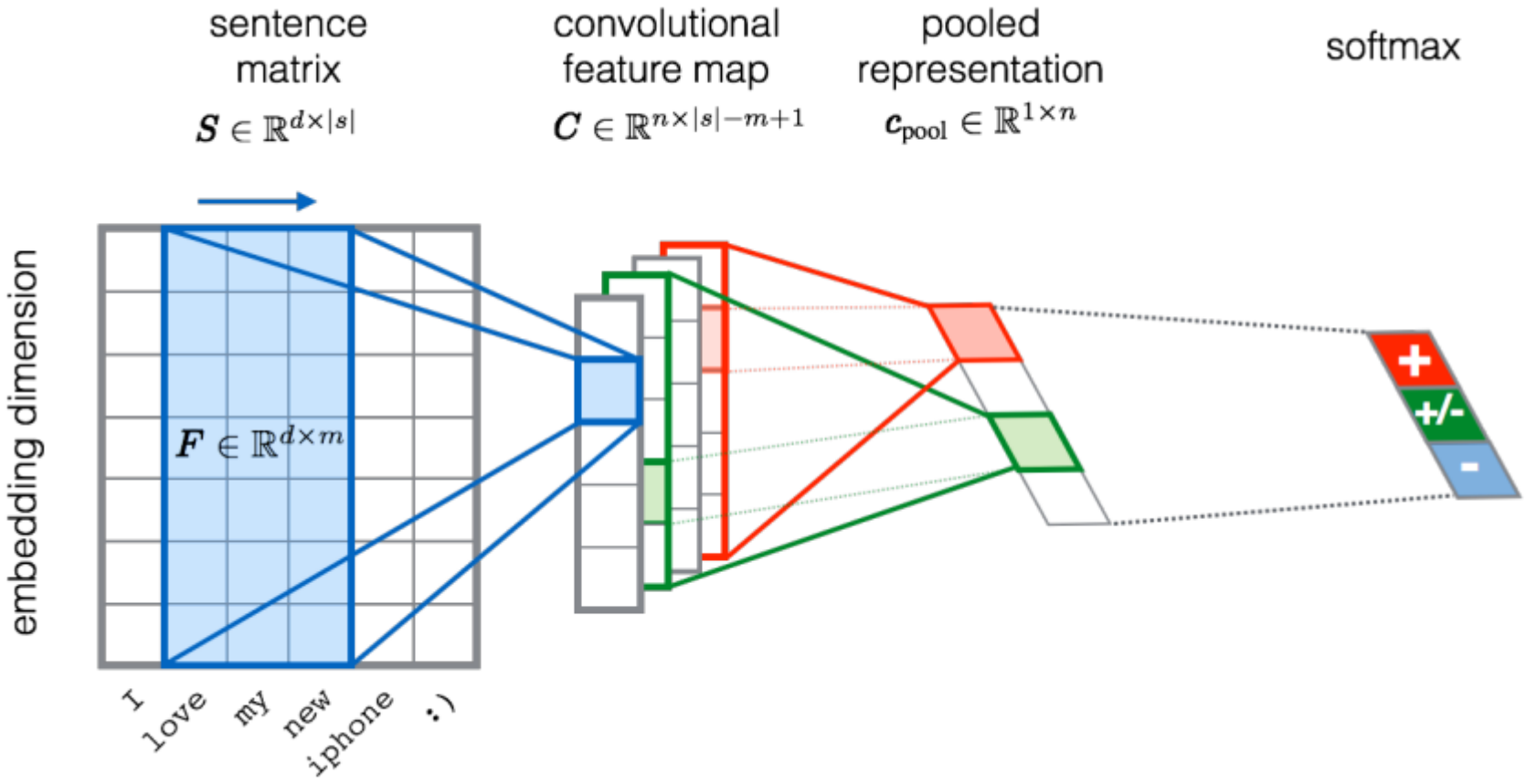


CNN for text classification



Zhang and Wallace (2015) A Sensitivity Analysis of (and Practitioners' Guide to) Convolution Neural Networks for Sentence Classification
https://arxiv.org/pdf/1510.03820.pdf
(follow on paper, not famous, but a nice picture)

CNN for text classification



Severyn, Aliaksei, and Alessandro Moschitti. "Twitter sentiment analysis with deep convolutional neural networks." *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2015.

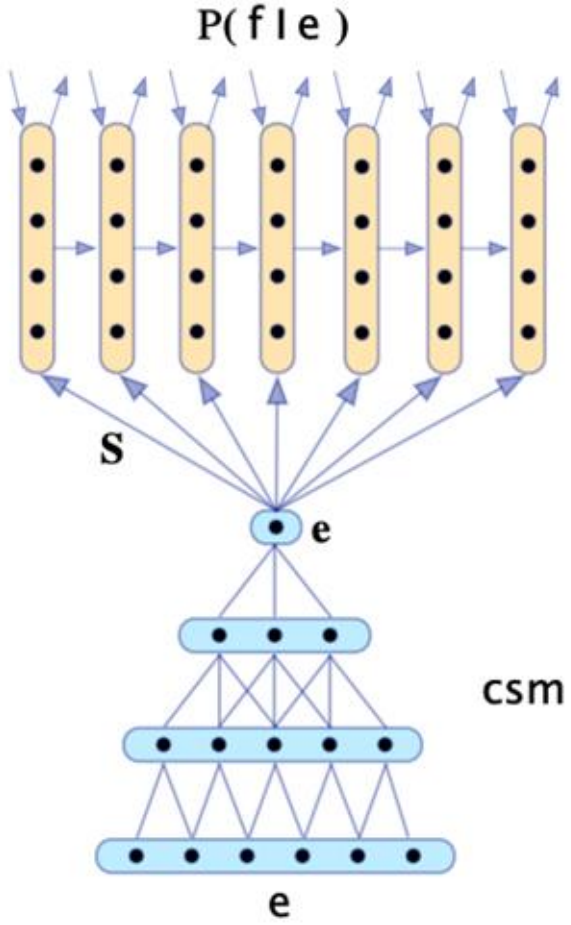
Batch Normalization (BatchNorm)

- Often used in CNNs
- Transform the convolution output of a batch by scaling the activations to have zero mean and unit variance
  - This is the familiar Z-transform of statistics
  - But updated per batch so fluctuation don't affect things much
- Use of BatchNorm makes models much less sensitive to parameter initialization, since outputs are automatically rescaled
  - It also tends to make tuning of learning rates simpler
- PyTorch: nn.BatchNorm1d

Ioffe and Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. http://proceedings.mlr.press/v37/ioffe15.pdf

CNN application: Translation

- One of the first successful neural machine translation efforts
- Uses CNN for encoding and RNN for decoding



Kalchbrenner and Blunsom (2013). "Recurrent Continuous Translation Models". https://www.aclweb.org/anthology/D13-1176

Part 4

Text -> Tokens

Why do we need tokenization?

- Tokenization is the first step in any NLP pipeline.
- A tokenizer breaks unstructured data and natural language text into chunks of information that can be considered as discrete elements.
  - Can be used directly as a vector representing that text.
  - Can also be used directly by a computer to trigger useful actions and responses.
  - Can be used in a machine learning pipeline as features.

https://neptune.ai/blog/tokenization-in-nlp

From Text to Tokens

- Many NLP models assume the input text has been tokenized and encoded as numerical vectors.
- There are several tokenization strategies we can adopt:
  - **Character Tokenization**: The simplest tokenization scheme is to feed each character individually to the model.
  - **Word Tokenization**: split into words and map each word to an integer.
  - **Subword Tokenization**: combine the best aspects of character and word tokenization.

Natural Language Processing with Transformers (NLPT), Chapter 2

Character tokenization

- Character-level tokenization:

```
text = "Tokenizing text is a core task of NLP."
tokenized_text = list(text)
print(tokenized_text)
```

```
['T', 'o', 'k', 'e', 'n', 'i', 'z', 'i', 'n', 'g', ' ', 't', 'e', 'x', 't', ' ',
'i', 's', ' ', 'a', ' ', 'c', 'o', 'r', 'e', ' ', 't', 'a', 's', 'k', ' ', 'o',
'f', ' ', 'N', 'L', 'P', '.']
```

- Ignores any structure in the text and treats the whole string as a stream of characters.
- Helps deal with misspellings and rare words,
- But the main drawback is that linguistic structures such as words need to be learned from the data. This requires significant compute, memory, and data.

- For this reason, character tokenization is rarely used in practice.
  - Need to preserve some structure of the text during tokenization.

Word Tokenization

- Split the input text into words and map each word to an integer.
- One simple class of word tokenizers uses whitespace to tokenize the text.

```
tokenized_text = text.split()
print(tokenized_text)

['Tokenizing', 'text', 'is', 'a', 'core', 'task', 'of', 'NLP.']
```

- Reduces the complexity of the training process.
- Punctuation is not accounted for, so ‘NLP.’ is treated as a single token.
- Given that words can include declinations, conjugations, or misspellings, the size of the vocabulary can easily grow into the millions!

Potential problem of a large vocabulary

- Requires neural networks to have an enormous number of parameters; expensive to train, and larger models are more difficult to maintain.
- A common approach is to limit the vocabulary and discard rare words by considering, say, the 100,000 most common words in the corpus.
  - Words that are not part of the vocabulary are classified as “unknown” and mapped to a shared UNK token.
  - We lose some potentially important, since the model has no information about words associated with UNK.
- Is there a compromise between character and word tokenization that preserved all the input information and some of the input structure?

Subword tokenization

- Basic idea: combine the best aspects of character and word tokenization
  - We want to split rare words into smaller units to allow the model to deal with complex words and misspellings.
  - We want to keep frequent words as unique entities so that we can keep the length of our inputs to a manageable size.

- Learned from the pre-training corpus using a mix of statistical rules and algorithms.

- The word pieces may be full words or parts of words.

Subword tokenization

- One of the main tools they utilize is breaking off affixes.
  - Because prefixes, suffixes, and infixes change the inherent meaning of words, they can also help programs understand a word's function.
  - This can be especially valuable for out of vocabulary words, as identifying an affix can give a program additional insight into how unknown words function.
- The sub word model will search for these sub words and break down words that include them into distinct parts.

"What is the tallest building"

→'what' 'is' 'the' 'tall' 'est' 'build' 'ing'

A blog about NLP tokenization

How does subword tokenization help the issue of OOV words

- A more complicated word, like ‘machinating’ (the present tense of verb ‘machinate’ which means to scheme or engage in plots).
- It’s unlikely that machinating is a word included in many basic vocabs.
  - Word tokenization: an unknown token.
  - Subword tokenization: an ‘unknown’ token and an ‘ing’ token.
- From there it can make valuable inferences about how the word functions in the sentence.
  - Either functioning as a verb turned into a noun, or as a present verb.
  - This dramatically narrows down how the unknown word, ‘machinating,’ may be used in a sentence.

A blog about NLP tokenization

Subword modeling

- Subword modeling in NLP has a wide range of methods for reasoning about structure below the word level. (Parts of words, characters, bytes.)
- The dominant modern paradigm is to learn a vocabulary of parts of words (subword tokens).
- At training and testing time, each word is split into a sequence of known subwords.
- Byte pair encoding (BPE) is a simple, effective strategy for defining a subword vocabulary.

Sennrich et al. (2016)

Byte Pair Encoding (BPE)

1. Start with a vocabulary containing only characters and an "end-of-word" symbol.

2. Using a corpus of text, find the most common adjacent characters "a, b"; add "ab" as a subword.

3. Replace instances of the character pair with the new subword; repeat until desired vocab size.

```
for i in range(num_merges):
  pairs = get_stats(vocab)
  best = max(pairs, key=pairs.get)
  vocab = merge_vocab(best, vocab)
  print(best)
```

Sennrich et al. (2016)

Byte Pair Encoding (BPE)

- Doing 8k merges => vocabulary of around 8000 word pieces.
- Includes many whole words
- Most SOTA neural machine translation (NMT) systems use this on both source + target.
- Now a similar method (WordPiece) is used in pretrained models.

Sennrich et al. (2016)

Tokenization Today

- All pretrained models use some kind of subword tokenization with a tuned vocabulary; usually between 50k and 250k pieces (larger number of pieces for multilingual models).
- As a result, classical word embeddings like GloVe are not used. All subword representations are randomly initialized and learned in the Transformer models.
- When using pretrained models, it is really important to make sure that you use the same tokenizer that the model was trained with.
- From the model's perspective, switching the tokenizer is like shuffling the vocabulary.
- If everyone around you started swapping random words like "house" for "cat," you'd have a hard time understanding what was going on too!

Stopwords/Punctuation Today

- In traditional methods for building a co-occurrence matrix and applying techniques like Singular Value Decomposition (SVD) for dimensionality reduction, it's common to remove both punctuation and stopwords.

- However, many models rely heavily on the context of each word in a sentence. This means that even seemingly insignificant words like stop words and punctuation can be important for understanding the full context of a sentence. Therefore, they are retained during tokenization.

- It's important to note that the specific preprocessing steps can vary depending on the task and the dataset. For instance, in certain cases, you might want to retain certain stopwords if they are relevant to the context of your analysis.

DistilBERT tokenizer in Hugging Face

```
from transformers import AutoTokenizer

model_ckpt = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_ckpt)
```

Automatically retrieve model configuration, pretrained weights, or vocab from the checkpoint

```
from transformers import DistilBertTokenizer

distilbert_tokenizer = DistilBertTokenizer.from_pretrained(model_ckpt)
```

Load the specific tokenizer manually

Text = ”Tokenizing text is a core task of NLP.”

```
encoded_text = tokenizer(text)
print(encoded_text)
```

How this tokenizer works

```
{'input_ids': [101, 19204, 6026, 3793, 2003, 1037, 4563, 4708, 1997, 17953,
2361, 1012, 102], 'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]}
```

```
tokens = tokenizer.convert_ids_to_tokens(encoded_text.input_ids)
print(tokens)
```

Convert them back into tokens

```
['[CLS]', 'token', '##izing', 'text', 'is', 'a', 'core', 'task', 'of', 'nl',
'##p', '.', '[SEP]']
```

Tokenizing the Whole Dataset

Special Token	[PAD]	[UNK]	[CLS]	[SEP]	[MASK]
Special Token ID	0	100	101	102	103

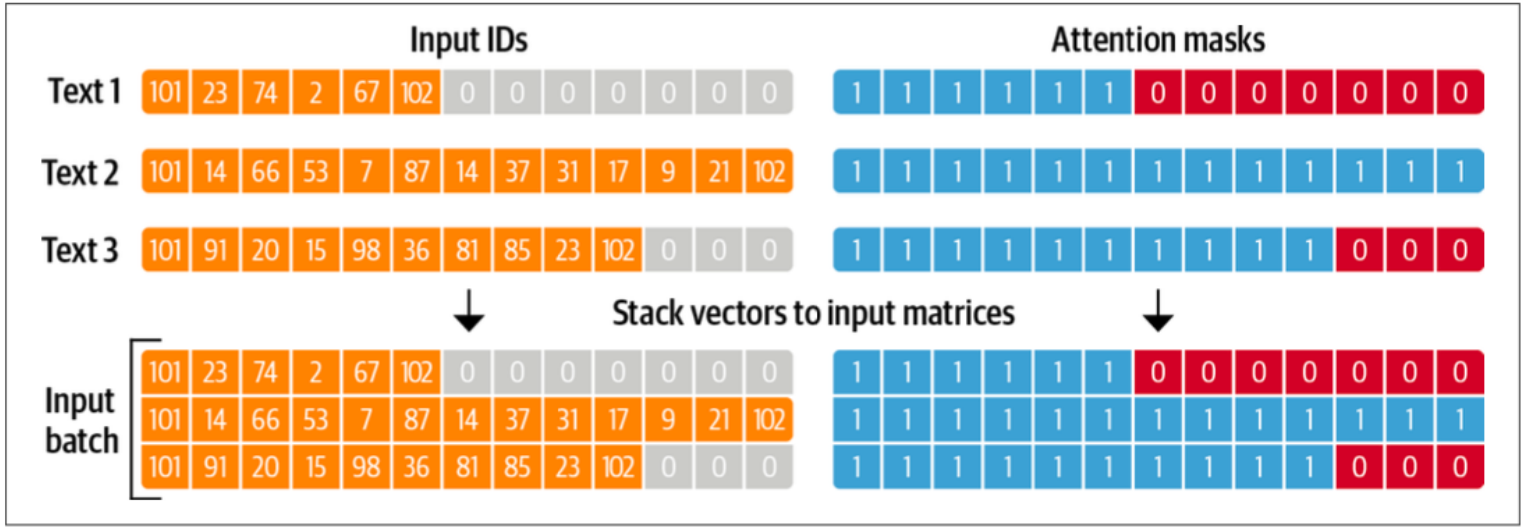


Figure 2-3. For each batch, the input sequences are padded to the maximum sequence length in the batch; the attention mask is used in the model to ignore the padded areas of the input tensors

All sequences have the same length, and you can efficiently process them in batches. The padding tokens are typically ignored by the network during computation.

Natural Language Processing with Transformers (NLPT), Chapter 2

Popular tokenization tools and libraries

- **NLTK (Natural Language Toolkit):**
  - A comprehensive library for natural language processing in python.
  - Includes various tokenization methods, including word tokenizers and sentence tokenizers.

- **Spacy:**
  - A popular library for natural language processing that provides high-performance tokenization along with many other NLP functionalities.

- **Tokenizer (from Hugging Face's Transformers):**
  - Part of the Hugging Face Transformers library, this tokenizer is specifically designed for use with modern transformer-based models like BERT, GPT, etc.

Readings

- **CH9 DL; CH 13 NNLP**
- **CH1-2 NLPT**
- LeCun et al. Gradient-based learning applied to document recognition. 1998
- Sennrich et al. Neural Machine Translation of Rare Words with Subword Units, 2016
- More details and implementation of BPE: https://huggingface.co/learn/nlp-course/chapter6/5?fw=pt
- More about WordPiece tokenization: https://huggingface.co/learn/nlp-course/chapter6/6?fw=pt